

آموزش صفر تا صد — نسخه سپتامبر ۲۰۲۶

Docker

از صفر مطلق تا دیپلوی واقعی

بسته‌بندی، اجرا و توزیع پروژه‌های Next.js در کانتینر

Compose v2

Multi-stage

Dockerfile

Docker Engine 29

Deploy on VPS

Volumes

۱۳ فصل · استک کامل Next.js + PostgreSQL + Prisma · تمرین با جواب · ۱۲۰ ارور کلاسیک

فهرست مطالب

نقشه دوره و زمین تمرین	شروع دوره	1
vs VM کانتینر، image/container/registry	فصل ۱ – داکر چیست و چرا	2
سلامت، Docker Desktop، WSL2، تأیید	فصل ۲ – نصب و hello-world	3
run، -p، logs، ps، چرخه عمر	فصل ۳ – اجرای کانتینرها	4
Alpine، تگ‌ها، لایه‌ها، pull	فصل ۴ – ایمیج‌ها و Docker Hub	5
exec -it، env، inspect، چرخه دیباگ	فصل ۵ – داخل کانتینر و دیباگ	6
FROM/COPY/RUN/CMD، کش لایه‌ها	فصل ۶ – اولین Dockerfile	7
multi-stage، omit=dev، کاهش حجم	فصل ۷ – multi-stage و کاهش حجم	8
api+db، استک، YAML، up/down	فصل ۸ – Docker Compose	9
named volume، bind mount، بکاپ	فصل ۹ – داده ماندگار	10
hostname، سرویس‌ها، DNS داخلی	فصل ۱۰ – شبکه کانتینرها	11
Next.js + Postgres + Prisma در عمل	فصل ۱۱ – استک Next.js + Postgres + Prisma 🚀	12
push، GHCR، VPS، امنیت	فصل ۱۲ – push و دیپلوی	13
مرور سریع + ۱۲ ارور کلاسیک	فصل ۱۳ – چیت‌شیت و گلاساری	14

دوره جامع Docker – از «داکر اصلاً چیه؟» تا دپلوی واقعی 🎓

FORMAT

PDF + MARKDOWN

LANGUAGE

فارسی

GENERATED BY

GLM-5.3-FLASH

LICENSE

MIT

🤖 تولید شده توسط هوش مصنوعی: این دوره به طور کامل توسط مدل GLM-5.3-Flash تولید و تدوین شده

است. نسخه دوره: سپتامبر ۲۰۲۶ – سازگار با Docker Desktop 4.9x / Engine 29 و Docker Compose v2 (compose) مخاطب: توسعه دهندگان وب و نرم افزار (به ویژه کار با Node.js / JavaScript / Next.js) که قصد دارند داکر را از صفر به صورت اصولی و کاربردی یاد بگیرند.

🎯 چرا داکر و چرا الان؟

مشکل کلاسیک در توسعه نرم افزار: «روی سیستم من کار می کرد، روی سرور یا سیستم بقیه کار نمی کنه!» – تفاوت نسخه های Node.js، سیستم عامل های مختلف، نسخه های متناقض دیتابیس یا تداخل پورت ها و پکیج ها. داکر دقیقاً برای پایان دادن به این ناهماهنگی ها ساخته شده است.

مزایای کلیدی یادگیری داکر: 1. یکپارچه سازی محیط توسعه و پروداکشن: کد در هر محیطی دقیقاً به یک شکل و در یک کانتینر ایزوله اجرا می شود. 2. کاربرد مستقیم در پروژه های واقعی: راه اندازی سریع دیتابیس ها و سرویس های مختلف بدون نیاز به نصب دستی روی سیستم عامل میزبان، و آماده سازی آسان پروژه برای کار تیمی یا دپلوی روی سرور.

پس از پایان این دوره: - اجرای استک های چندسرویسی با `docker compose up` به یک کار روتین و ساده تبدیل می شود. - امکان کانتینر سازی برنامه های تحت وب و دپلوی استاندارد آن ها روی هر نوع سروری فراهم خواهد شد. - توانایی کامل در نوشتن، بهینه سازی و دیباگ فایل های `Dockerfile` و `compose.yaml` به دست می آید.

فصل	موضوع	چی یاد می‌گیری
فصل ۱	داکر چیست؟	image/container/registry، کانتینر vs ماشین مجازی
فصل ۲	نصب	Docker Desktop روی ویندوز، WSL2، اولین hello-world
فصل ۳	اجرای کانتینر	run، پورت‌ها (p-)، stop/rm، ps، logs
فصل ۴	ایمیج‌ها	pull، تگ‌ها، Docker Hub، لایه‌ها و کش
فصل ۵	داخل کانتینر	exec و شل، env، inspect – دیباگ واقعی
فصل ۶	اولین Dockerfile	بسته‌بندی یک اپ Node – خط به خط
فصل ۷	ایمیج تمیز	multi-stage، alpine، کش لایه‌ها، کاهش حجم
فصل ۸	★ Docker Compose	کل استک با یک فایل YAML و یک دستور
فصل ۹	داده ماندگار	volumes، bind mounts، workflow توسعه زنده
فصل ۱۰	شبکه‌ها	کانتینر به کانتینر؛ اسم سرویس = hostname
فصل ۱۱	🚀 داکر در کار روزمره	استک کامل: Next.js + Postgres + Prisma در کانتینر
فصل ۱۲	push و دپلوی	Docker Hub/GHCR، دپلوی روی VPS، پاکسازی و امنیت
فصل ۱۳	چیت‌شیت و گلاساری	مرور سریع + ۱۲ ارور کلاسیک داکر

زمین تمرین

همه تمرین‌ها با داکر واقعی انجام می‌شن – کانتینرها کاملاً ایزوله‌اند، بنابراین با خیال راحت آزمایش کنید:

```
mkdir ~/docker-playground && cd ~/docker-playground
```

قانون طلایی داکر: کانتینر = یک بسته نرم‌افزاری ایزوله و یکبارمصرف. در صورت بروز هرگونه مشکل، با `docker rm` پاک شده و مجدداً از نو راه‌اندازی می‌شود.

ابزار / مهارت	کاربرد	دستور بررسی
ویندوز 10/11 ۶۴بیتی (یا لینوکس/مک)	بستر اجرای Docker Desktop یا Engine	—
آشنایی مقدماتی با ترمینال / شل	اجرای دستورات داکر در خط فرمان	<code>bash --version</code>
نصب بودن Node.js (برای مثال‌های عملی)	اجرای کدهای نمونه وب	<code>node -v</code>
Git	کلون و مدیریت مخازن پروژه‌ها	<code>git --version</code>

💡 در طول این دوره مفاهیم خط فرمان، متغیرهای محیطی (`.env`) و دستورات روتین شل به‌طور عملی در خدمت مدیریت کانتینرها به کار گرفته می‌شوند و تمام مراحل گام‌به‌گام توضیح داده شده است.

تبدیل به PDF

نسخه PDF رنگی راست‌به‌چپ (فونت وزیرمتن) در همین پوشه آماده است: «دوره کامل Docker.pdf». برای بازسازی: `python build/build_pdf.py` + رندر Chrome headless (دستور در پرامپت آماده پوشه والد).

کلمات کلیدی و برچسب‌های سئو (SEO Keywords & Tags)

تگ‌های پیشنهادی برای مخزن گیت‌هاب (Topics): `docker`, `docker-compose`, `containerization`, `devops`, `dockerfile`, `microservices`, `persian-tutorial`, `farsi`, `دوآپس`, آموزش-داکر

کلمات کلیدی جستجو:

آموزش جامع داکر Docker به زبان فارسی، کتاب آموزش داکر pdf، داکر کامپوز برای توسعه دهندگان، کانتینرسازی پروژه های وب، آموزش دوآپس از صفر

دانلود مستقیم نسخه چاپی و PDF کتاب

برای دسترسی و دانلود مستقیم فایل PDF کامل این دوره آموزشی، به بخش **Releases** همین ریپازیتوری مراجعه کنید یا فایل PDF قرار داده شده در ریشه مخزن را دریافت نمایید.

فصل ۱ – داکر چیست و چرا همه دنیا ازش استفاده می‌کنند؟

🎯 هدف این فصل: بدون نصب هیچ‌چیز، مدل ذهنی درست رو بسازیم: کانتینر چیه، فرقی با ماشین مجازی، و سه کلمه‌ی image/container/registry که کل داکر همینه. هنوز هیچ دستوری نمی‌زنیم.

۱.۱ – مسئله: «روی سیستم من کار می‌کنه!» 🤔

یک برنامه کاربردی فقط از چند فایل سورس‌کد تشکیل نشده است – بلکه به یک محیط اجرایی کامل نیاز دارد:

- نسخه مشخصی از ران‌تایم (مثل Node.js یا پایتون)
- پکیج‌ها و کتابخانه‌ها با نسخه‌های هماهنگ
- سرویس دیتابیس با نسخه و تنظیمات معین
- متغیرهای محیطی صحیح

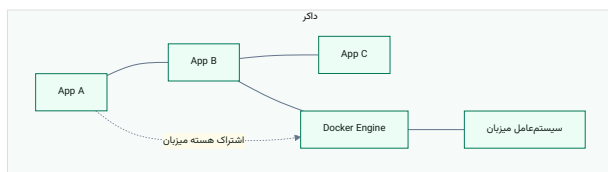
به این مجموعه، **محیط و زیرساخت اجرا (Runtime Environment)** می‌گویند. چالش اصلی: محیط هر رایانه متفاوت است – سیستم توسعه‌دهنده، لپ‌تاپ سایر اعضای تیم، سرور Production، سرور تست CI/CD... تفاوت در این محیط‌ها منجر به خطاهای غیرمنتظره می‌شود.

راهکار داکر: برنامه را همراه با تمام پیش‌نیازها و محیط اجرایی‌اش بسته‌بندی کنید – همانند قرار دادن یک سرویس کامل به همراه تنظیمات در یک کانتینر ایزوله. این کانتینر در هر بستری که اجرا شود، رفتاری کاملاً یکسان و قابل پیش‌بینی خواهد داشت: یک بار بیلد کنید، در هر محیطی اجرا کنید.

۱.۲ – کانتینر در برابر ماشین مجازی (سوال شماره یک مصاحبه‌ها!)

هر دو «ایزوله‌سازی» می‌دن ولی از دو مسیر کاملاً متفاوت:

کانتینر	ماشین مجازی (VM)	
فقط فضای کاربر	کل سخت‌افزار + یک سیستم‌عامل کامل	چی رو مجازی می‌کنه؟
ده‌ها مگابایت	گیگابایت‌ها	اندازه
کمتر از یک ثانیه ⚡	ده‌ها ثانیه تا دقیقه	زمان راه‌اندازی
ناچیز	سنگین	مصرف رم/CPU
ده‌ها تا	شاید ۲-۳ تا	چندتا روی لپ‌تاپ؟



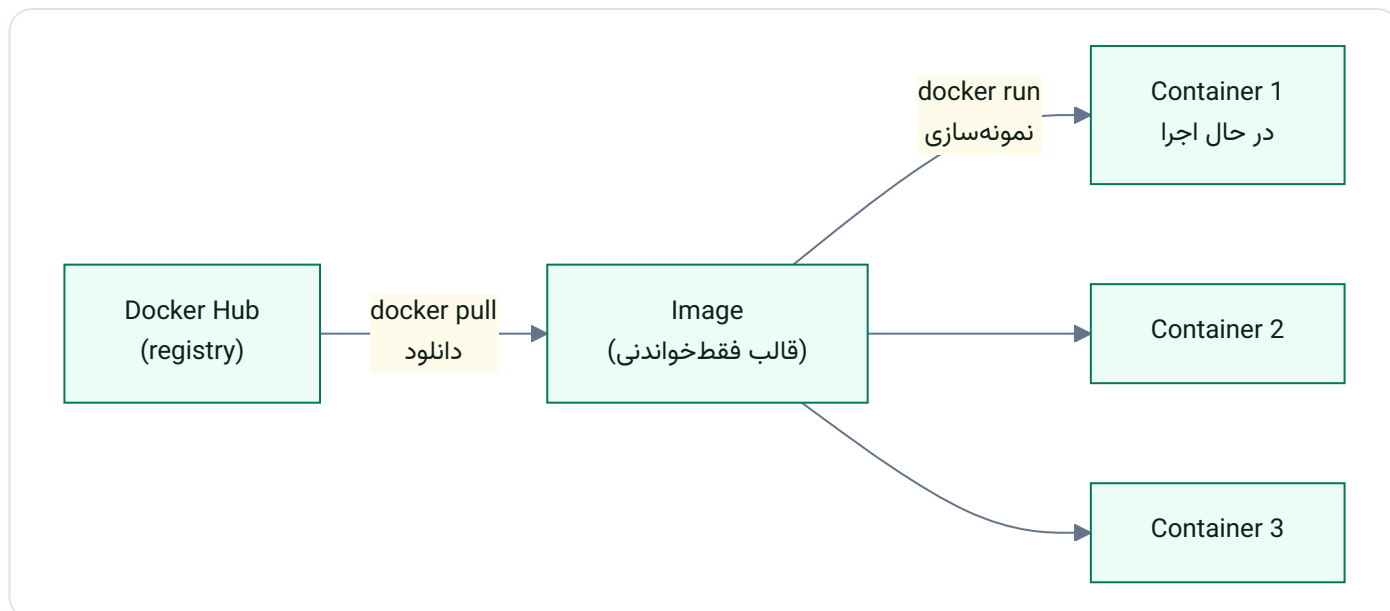
راز سبکی کانتینر: کانتینرها **هسته‌ی (kernel) سیستم عامل میزبان رو به اشتراک می‌ذارن** و فقط فایل‌ها و پروسه‌های خودشون رو ایزوله می‌کنن. VM یک OS کامل میاره بالا؛ کانتینر فقط «یک اتاق» داخل OS فعلی می‌سازه.

💡 نکته مهم درباره ویندوز: کانتینرهای لینوکسی (که بخش اعظم دنیای کانتینرها را تشکیل می‌دهند) روی ویندوز هم اجرا می‌شوند – زیرا Docker Desktop یک لینوکس فوق‌العاده سبک را در پس‌زمینه با WSL2 مدیریت می‌کند (فصل ۲). بنابراین کانتینری که روی سیستم محلی اجرا می‌شود، دقیقاً همان لینوکس واقعی است که روی سرورها اجرا خواهد شد.

۱.۳ – سه کلمه طلایی: Image، Container، Registry

کل داکر در این سه مفهوم خلاصه می‌شود:

مفهوم	چیه	شبیه به (دنیای برنامه‌نویسی)
Image (ایمیج)	بسته‌ی فقط‌خواندنی حاوی اپ + وابستگی‌ها + محیط – قالب آماده	کلاس (Class)
Container (کانتینر)	نمونه‌ی در حال اجرای یک ایمیج	نمونه ساخته‌شده (Instance / Object)
Registry (رجیستری)	پایگاه ذخیره و توزیع ایمیج‌ها (Docker Hub)	مخازن پکیج مانند npmjs یا PyPI



مشابه مفهوم کلاس و شیء در شیء‌گرایی: از یک ایمیج `postgres:18` می‌توان ده کانتینر مستقل (ده دیتابیس کاملاً ایزوله) ایجاد کرد. ایمیج اصلی ثابت می‌ماند و کانتینرها چرخه حیات مستقل دارند. چرخه‌ی کار نیز بسیار شبیه به پکیج‌منیجرهاست:

مدیریت پکیج (npm/pip)	داکر (Docker)
<code>npm install axios</code> (دانلود پکیج)	<code>docker pull postgres:18</code> (دانلود ایمیج)
اجرای کد پکیج	<code>docker run postgres:18</code> (اجرای کانتینر)
<code>npm publish</code> (انتشار پکیج)	<code>docker push myapp:1.0</code> (انتشار ایمیج)

🔑 درک منطق پکیج‌منیجرها به سادگی به درک داکر کمک می‌کند – تنها تفاوت این است که به جای یک کتابخانه کد، کل محیط اجرایی جابه‌جا می‌شود.

۱.۴ – داکر چیه دقیقاً؟

Docker = مجموعه ابزاری استاندارد برای ساختن، اجرا و توزیع کانتینرها. شامل:

نقش	جزء
موتور اجرای کانتینرها (daemon پشت صحنه + هسته داکر)	Docker Engine
ابزار خط فرمان <code>docker</code> جهت ارسال دستورات به Engine	Docker CLI
اپ ویندوزی/مکی که Engine + GUI + WSL2 رو یکجا نصب می‌کنه	Docker Desktop
مدیریت چند کانتینر با یک فایل YAML (فصل ۸)	Docker Compose
بزرگ‌ترین رجیستری عمومی	Docker Hub

مهم‌ترین نکته: داکر اساساً مبتنی بر رابط خط فرمان (CLI) است – `docker ps` ، `docker run` ، `docker pull` . تمام این دستورات از یک الگوی مشخص و منطقی پیروی می‌کنند: `docker <فعل> <موضوع> [<فلگ‌ها>]` – مثل `docker image` ، `docker container ls` ، `pull` و غیره.

۱.۵ – چرا شرکت‌ها عاشقشان

- **یکنواختی محیط:** `dev = staging = production` – همون ایمیج!
- **مستقل‌سازی تیم‌ها:** بدون نیاز به نصب دستی Postgres و Redis و Kafka روی سیستم میزبان، فقط با یک دستور اجرا می‌شوند (فصل ۸)
- **CI/CD:** پایپ‌لاین‌های گیت‌هاب اکشنز یا GitLab برنامه را داخل کانتینر تست و بیلد می‌کنند – محیطی کاملاً پاک و ایزوله در هر بار اجرا
- **دپلوی ساده:** یک ایمیج یکسان روی هر نوع سرور و ابری (فصل ۱۲)
- **مقیاس‌پذیری:** افزایش ترافیک؟ از همان ایمیج چندین کانتینر به صورت موازی بالا می‌آید

✓ جمع‌بندی فصل

- مشکل: تفاوت محیط‌ها. جواب: بسته‌بندی اپ با **محیطش** = کانتینر
- کانتینر $\neq$ VM: بدون OS کامل، سبک، ثانیه‌ای بالا
- **Image = کلاس، Container = آبجکت، Registry = npm برای ایمیج**
- داکر = Engine + CLI + Desktop + Compose + Hub؛ و یک ابزار استاندارد خط فرمان است
- چرا شرکت‌ها: یکنواختی، ایزوله‌سازی، دپلوی، مقیاس

📄 تمرین فصل ۱

۱. فرق image و container رو با analogy/آبجکت تو یک جمله بگو.

2. چرا کانتینر سبک‌تر از VM است؟ (در یک جمله: کی kernel رو شریک می‌شه؟)

3. جدول معادل‌ها رو کامل کن: `docker push` ↔ ؟ ، ؟ ↔ `docker run` ، ؟ `npm` ↔ `docker pull`

4. یک موقعیت واقعی از توسعه نرم‌افزار بنویسید که داکر می‌تواند از بروز ناهماهنگی نسخه‌ها جلوگیری کند (مثل تداخل نسخه‌های مختلف پایگاه داده یا پکیج‌های ناسازگار).

▼ جواب‌ها

3. `npm install` ↔ `docker pull` ؛ `docker run` ↔ اجرای برنامه‌ی نصب‌شده (مثل `npx`) ؛ `docker push` ↔ `npm publish`.

➔ **فصل بعد:** نصب Docker Desktop روی ویندوز و اولین hello-world دنیای کانتینرها!

فصل ۲ – نصب داکر روی ویندوز و اولین hello-world

🎯 هدف: Docker Desktop رو درست نصب کنی (با WSL2)، مطمئن شی همه چیز وصل است، و اولین کانتینر رو اجرا کنی. آخر فصل `docker run hello-world` برای تو یک دستور جادویی نیست.

۲.۱ – پیش‌نیاز ویندوز: WSL2 چیست و چرا لازمه؟

کانتینرهای دنیا **لینوکسی‌ان** و به هسته‌ی لینوکس نیاز دارن. ویندوز هسته لینوکس نداره – پس Docker Desktop یک لینوکس سبک با (Windows Subsystem for Linux 2) **WSL2** پشت صحنه بالا میاره و Engine اونجا اجرا می‌شه. تو احساسش نمی‌کنی؛ فقط بدون که داری کانتینر لینوکس واقعی اجرا می‌کنی.

چک/فعال‌سازی WSL2 (یک بار، در PowerShell با دسترسی ادمین):

```
wsl --status      # اگر نصب است: نسخه پیش‌فرض باید 2 باشد
wsl --install      # اگر نیست: نصب (ویندوز ری‌استارت می‌خواهد)
```

💡 گاهی پیام «Virtualization not enabled» می‌بینی – یعنی باید در BIOS سیستم، / VT-x Virtualization (AMD-V) فعال شه. اکثر سیستم‌های جدید روشنه.

۲.۲ – نصب Docker Desktop

1. برو به <https://www.docker.com/products/docker-desktop/> → نسخه ویندوز (AMD64) رو بگیر (4.9x فعلی)
2. نصب‌کننده رو اجرا کن – گزینه **Use WSL 2 instead of Hyper-V** تیک‌خورده بمونه
3. بعد از نصب و ری‌استارت، Docker Desktop رو باز کن – آیکون نهنگ 🐳 در tray باید ثابت (بدون انیمیشن) بمونه یعنی Engine بالااست
4. قرارداد لایسنس رو قبول کن (برای استفاده شخصی و تیم‌های کوچک رایگانه)

۲.۳ – تأیید سلامت: چهار دستور

در Git Bash (یا ترمینال VS Code):

```
$ docker version
Client: Docker Engine - Community
Version: 29.x.x
Server: Docker Engine - Community
Engine Version: 29.x.x
✓ (وصل است Engine به CLI هم باید باشد - یعنی Server مهم: بخش)

$ docker info | head -20 # اطلاعات کامل محیط

$ docker run hello-world # اولین کانتینر
Hello from Docker!
...

$ docker compose version
Docker Compose version v2.x.x
```

چیزی که تازه اتفاق افتاد، قدم به قدم: 1. `docker run hello-world` → داکر دید ایمج `hello-world` لوکال نیست 2. از `pull` Docker Hub کرد (دانلود چند کیلوبایت) 3. ازش یک کانتینر ساخت و اجرا کرد 4. کانتینر کارش رو کرد (پیام چاپ کرد) و تموم شد این چرخه «نگردم بگیر → اجرا کن» همون npm-like رفتاریه که فصل ۱ گفتم.

۲.۴ – آشنایی با Docker Desktop (GUI)

ترمینال ابزار اصلی ماست ولی GUI هم برای «دیدن» خوبه – تب‌های مهمش:

- **Containers**: لیست کانتینرهای در حال اجرا/خاموش + دکمه **Logs** + start/stop/delete گرافیکی
- **Images**: ایمج‌های دانلودشده روی سیستم
- **Volumes**: داده‌های ماندگار (فصل ۹)

💡 قانون دوره: هر کاری رو *اول* با CLI یاد بگیر (چون در شرکت/سرور فقط CLI داری)، بعد GUI رو به چشم «داشبورد نگاه سریع» استفاده کن.

۲.۵ – تنظیمات پیشنهادی اولیه

Docker Desktop → Settings (چرخ‌دنده):

تنظیم	پیشنهاد	چرا
General → Start Docker Desktop when you sign in	به سلیقه	اگه هر روز کار می‌کنی، روشن بذار
Resources → Advanced (در WSL2 محدود است)	پیش‌فرض WSL2	WSL2 خودش مدیریت می‌کنه
...Choose how to configure installs	پیش‌فرض	—

و یک نکته ترمینالی: مطمئن شو `docker` در ترمینال (مثل PowerShell، Git Bash یا CMD) کار می‌کنه — Docker Desktop در زمان نصب مسیر اجرایی خود را به متغیر محیطی PATH سیستم اضافه می‌کند تا در هر مسیری قابل فراخوانی باشد. اگر ابتدا کار نکرد، کافیه پنجره ترمینال را ببندید و مجدداً باز کنید تا PATH بازخوانی شود.

۲.۶ – عیب‌یابی‌های نصب رایج

مشکل	علاج
<code>docker: command not found</code>	Docker Desktop روشن نیست یا PATH تازه نشده — ترمینال جدید باز کن
Docker Desktop منتظر Engine می‌مونه	WSL2 را به‌روز کن: <code>wsl --update</code> (PowerShell ادمین) و ری‌استارت
«WSL 2 installation is incomplete»	همون <code>wsl --update</code> + ری‌استارت ویندوز
کندی عجیب	آیکون نهنگ را چک کن — Engine خاموشه؛ Desktop را باز کن

✓ جمع‌بندی فصل

- WSL2 = لینوکس زیر داکر روی ویندوز؛ `wsl --install` و `wsl --update`
- Docker Desktop نصب شد؛ `docker version` باید Client و Server را نشان دهد
- `docker run hello-world` = چرخه کامل: pull از Hub → ساخت کانتینر → اجرا → پایان
- GUI فقط داشبورد؛ CLI ابزار اصلی تو است
- `command not found` = دسکتاپ خاموش یا ترمینال قدیمی

📄 تمرین فصل ۲

1. نصب را کامل کن و هر چهار دستور بخش ۲.۳ را بزن — خروجی `docker version` را باز کن و نسخه Engine را بنویس.
2. `docker run hello-world` را دوبار بزن. بار دوم چقدر سریع‌تر بود؟ چرا؟ (راهنمایی: ایمج لوکال شده — مثل npm `!cache`)
3. Docker Desktop را باز کن: در تب Images، ایمج `hello-world` را ببین. در تب Containers، کانتینر تمام‌شده را پیدا کن (وضعیتش چیه؟).
4. (کشف آزاد) `docker --help` را بزن و فقط اسم زیرفرمان‌های اصلی را نگاه کن — لیست طولانیه ولی از فصل بعد، این‌ها را همه می‌شناسی.

▼ جواب تمرین ۲

بار دوم فوری است چون ایمیج hello-world از قبل روی دیسک است و داکر فقط کانتینر از رویش می‌سازد – دانلودی در کار نیست. با `docker images` می‌توانی ایمیج لوکال را ببینی.

➔ **فصل بعد:** کانتینرهای واقعی – وب‌سرور اجرا کن، پورت‌ها را وصل کن و لاگ‌ها را بخوان.

فصل ۳ – اجرای کانتینرهای واقعی: run، پورت‌ها، لاگ‌ها

🎯 هدف: کار با nginx (وب‌سرور معروف) – چون رفتارش مثل هر سرویسیه که تو اجرا می‌کنی: یک پروسه، یک پورت، لاگ. آخر فصل `docker run` رو با فلگ‌های پرکاربردش مسلطی.

۳.۱ – اولین سرویس واقعی: nginx

nginx یک وب‌سرور (مثل Apache) که میلیون‌ها سایت باهاش کار می‌کنن – و ایمپج رسمی‌اش ۴۰ مگابایته. اجراش کن:

```
$ docker run -d -p 8080:80 --name my-web nginx
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
... (لایه‌ها دانلود می‌شوند) ...
a1b2c3d4e5f6...
```

مرورگر رو باز کن: `Welcome to nginx` – `http://localhost:8080`! رو می‌بینی. 🎉

فلگ‌ها رو باز کنیم (الگوی استاندارد دستور و گزینه‌ها در خط فرمان):

بخش	معنی
run	از روی ایمپج، کانتینر بساز و اجرا کن
-d	detached – اجرای کانتینر در پس‌زمینه (آزاد ماندن ترمینال)
-p 8080:80	port mapping – نگاشت پورت میزبان به کانتینر
--name my-web	اسم دلخواه برای کانتینر (در غیر این صورت یک نام تصادفی می‌گیرد)
nginx	اسم ایمپج مبنا (در صورت نبودن به صورت خودکار دانلود می‌شود)

۳.۲ – `-p 8080:80` : مپینگ پورت (مهم‌ترین فلگ!)

کانتینر یک **دنیای ایزوله** است – وب‌سرور داخلش روی پورت ۸۰ گوش می‌ده، ولی از مرورگر تو اصلاً دیده نمی‌شه، چون کانتینر شبکه خودش رو داره.

`-p 8080:80` یعنی: «پورت 8080 سیستم من (میزبان) → پورت 80 کانتینر»

مرورگر تو
localhost:8080

پورت میزبان

پورت مپینگ
80 → 8080

پورت کانتینر

nginx داخل کانتینر
گوش می‌دهد روی 80

چرا این جداسازی نعمته؟ چون می‌تونی ۵ تا nginx داری – هر کدوم پورت ۸۰ خودش رو دارن – و روی میزبان به 8080 و 8081 و 8082 وصل‌شون کنی. بدون تداخل! (دقیقاً همون دردی که با «پورت ۳۰۰۰ اشغاله» می‌کشیدی – داکر اینا رو مدیریت می‌کنه).

```
$ docker run -d -p 8081:80 --name web-two nginx      # دومی روی 8081
$ docker run -d -p 3000:3000 --name my-app node:22  # ها dev-server الگوی همه
```

⚠ سمت چپ (پورت میزبان) رو تو انتخاب می‌کنی و باید آزاد باشه؛ سمت راست (پورت کانتینر) رو خود برنامه تعیین می‌کنه – nginx همیشه ۸۰ است، Node app تو همون که کدت نوشته (مثلاً 3000).

۳.۳ – دیدن وضعیت: docker ps

```
$ docker ps
CONTAINER ID   IMAGE     COMMAND                  ...   PORTS          NAMES
a1b2c3d4e5f6   nginx    "/docker-entrypoint. ..." ...   0.0.0.0:8080->80/tcp   my-web

$ docker ps -a
# همه، حتی متوقف‌شده‌ها = -a
```

دستور `ps` (مخفف process status) وضعیت پروسه‌ها و کانتینرهای در حال اجرا را نمایش می‌دهد. با سوییچ `-a` می‌توانید لیست تمام کانتینرها حتی کانتینرهای متوقف‌شده را مشاهده کنید تا زمانی که با `docker rm` حذف شوند.

۳.۴ – لاگ‌ها: docker logs

سرور داخلی چی می‌گه؟

```
$ docker logs my-web          # همه لاگ‌ها تا الان
$ docker logs -f my-web       # follow -f (در لینوکس tail مشابه دستور)
$ docker logs --tail 20 my-web # فقط ۲۰ خط آخر
$ docker logs -f --tail 50 my-app # ترکیب رایج: ۵۰ خط آخر + ادامه زنده
```

این «لاگ‌خوانی» مهم‌ترین مهارت دیباگ داکر است: هر خروجی استاندارد که برنامه داخل کانتینر به `stdout` یا `stderr` ارسال می‌کند مستقیماً توسط داکر ذخیره شده و اینجا قابل مشاهده است. همچنین می‌توانید با دستوراتی مانند `grep` لاگ‌ها را فیلتر کنید:

```
$ docker logs my-app 2>&1 | grep -i error
```

۳.۵ – چرخه عمر: stop, start, restart, rm

```
$ docker stop my-web          # خاموش (کانتینر می‌ماند، فقط متوقف است)
$ docker start my-web         # دوباره روشن – داده‌هایش سرجایش است
$ docker restart my-web       # ری‌استارت
$ docker rm my-web            # rm -f: نشده، اول stop حذف کامل – با ⚠️
$ docker rm $(docker ps -aq)  # (!نگاه کن docker ps -a اول با) حذف همه کانتینرها
```

🔑 **mental model:** کانتینر مثل یک object موقتی – بساز، استفاده کن، دور بنداز. ایده‌ی «سرور مقدسی که می‌ترسم خرابش کنم» مال دنیای قدیمه. داده مهم رو نه توی کانتینر نگه می‌داریم، نه باهاش کار می‌کنیم بلکه توی **volume** نگه می‌داریم (فصل ۹).

میان‌بر مفید: `docker container prune` = حذف همه کانتینرهای متوقف‌شده (جاروی هفتگی).

۳.۶ – چند فلگ run دیگه که همین امروز لازم می‌شن

```
$ docker run -d --name db -e POSTGRES_PASSWORD=1234 postgres:18
# -e = ارسال متغیر محیطی (Environment Variable) برای تنظیمات اولیه

$ docker run -d --restart unless-stopped --name my-web nginx
# ری‌استارت خودکار – اگه کرش کرد یا سیستم ری‌استارت شد، داکر خودش بالا می‌آوردش

$ docker run --rm hello-world
# --rm = بعد از تمام شدن، خودش را حذف کن (برای ابزارهای یک‌بارمصرف عالی)
```

الگوی `-e` رو حفظ کن – ایمیج‌های رسمی (postgres, mysql, redis...) تنظیماتشون رو با `env` می‌گیرن؛ همون `.env` دنیای داکر.

✓ جمع‌بندی فصل

- الگوی طلایی: اسم ایمیج `name` --میزبان: کانتینر `docker run -d -p`
- `-p` پورت میزبان را به پورت کانتینر وصل می‌کند – چند نسخه بدون تداخل
- `docker ps` (در حال اجرا) / `ps -a` (همه) / `logs -f` (دنبال زنده)
- چرخه عمر: stop/start/restart/rm – کانتینر یک‌بارمصرف است؛ `prune` جارو
- `-e` = کانفیگ با متغیر محیطی؛ `--restart unless-stopped` = ماندگاری سرویس

📄 تمرین فصل ۳

1. nginx را با نام `web-one` روی پورت 8080 اجرا کن و در مرورگر ببینش.
2. یک nginx دوم با نام `web-two` روی 8081 اجرا کن – `docker ps` بزن و دو خط را با هم مقایسه کن.

3. لاگ `web-one` را بزن: اول معمولی، بعد با `-f` – و در مرورگر یکبار صفحه را refresh کن (خط access جدید در لاگ زنده می‌آید!).
4. `web-two` را stop کن؛ `docker ps -a` بزن و وضعیتش را ببین؛ start کن؛ بعد هر دو را کامل حذف کن.
5. چالش: `postgres:18` `POSTGRES_PASSWORD=1234` `pg` `--name` `5432:5432` `-d` `-p` `docker run` را اجرا کن – دیتابیس دورات در کانتینر بالا آمد! با `psql` وصل شو و بعدش خداحافظی کن. (فصل ۱۱ کاملش می‌کنیم).

▼ راهنمای تمرین ۵

```
docker run -d -p 5432:5432 --name pg -e POSTGRES_PASSWORD=1234 postgres:18
docker logs -f pg          # ببینی (چند ثانیه صبر) تا "database system is ready"
psql "postgresql://postgres:1234@localhost:5432/postgres" -c "SELECT version();"
docker rm -f pg
```

➔ **فصل بعد:** دنیای ایمیج‌ها – لایه‌ها، تگ‌ها و Docker Hub.

فصل ۴ – ایمیج‌ها: تگ‌ها، لایه‌ها و Docker Hub

🎯 هدف: ایمیج رو نه «یک فایل داندودی»، بلکه یک پشته‌ی لایه‌ای بفهمی – چون همه‌ی جادوی داکر (سرعت build، کش، اشتراک فضا) از همین لایه‌ها میاد. Plus: کار با Docker Hub مثل npm.

۴.۱ – ایمیج از کجا میاد؟ Docker Hub

node ، postgres ، nginx ، redis ، میلیون‌ها ایمیج آماده: Docker Hub (hub.docker.com) = npmjs ... python

```
$ docker search postgres # جستجو (مثل npm search)
$ docker pull postgres:18 # دانلود ایمیج
$ docker images # لیست ایمیج‌های لوکال ★
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
postgres	18	a1b2c3d4e5f6	2 weeks ago	431MB
nginx	latest	f6e5d4c3b2a1	3 weeks ago	188MB
hello-world	latest	9c7a54a9d43c	8 months ago	10.1kB

قانون انتخاب ایمیج برای استفاده (همون تفکر انتخاب پکیج npm): - اون‌هایی با تیک **Official Images** (ادامه‌ی library/) - نگهداری رسمی و امن - توضیحات + تعداد pull بالا - بروز بودن تگ‌ها

۴.۲ – تگ‌ها: نسخه‌های ایمیج ★

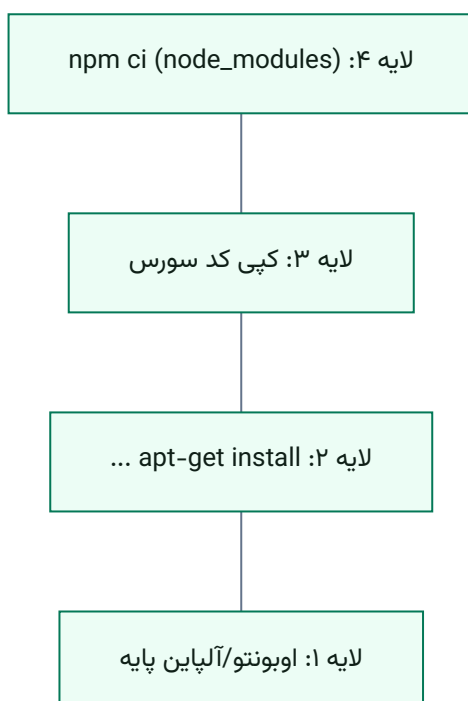
اسم کامل یک ایمیج: registry/owner/name:tag – در عمل:

مثال	معنی
postgres:18	نسخه 18 – ✓ توصیه‌شده برای کار واقعی
postgres:18.4	دقیق‌تر: نسخه 18.4
postgres:latest	«آخرین» – ⚠️ شناور! امروز ۱۸، ماه بعد شاید ۱۹
node:22-alpine	node نسخه 22، واریانت سبک alpine
postgres (بدون تگ)	= latest

🔔 **قانون حرفه‌ای:** در هر چیز جدی (compose فایل، Dockerfile، دپلوی) تگ دقیق بزن، نه `latest`. چون `latest` مثل `dependencies: "*" در npm است` – امروز کار می‌کنه، فردا بدون تغییر کدت می‌شکنه (breaking update خودکار!).

۴.۳ – لایه‌ها: راز ساختار ایمج

ایمج یک فایل یکپارچه نیست – پشته‌ای از لایه‌های فقط خواندنی است. هر دستور Dockerfile (فصل ۶) یک لایه جدید می‌سازد:



سه خاصیت طلایی لایه‌ها:

- ۱. کش (Cache):** داکر لایه‌های تغییر یافته رو از کش میاره – اگه فقط سورس عوض شد، لایه `npm ci` دوباره اجرا نمی‌شه (تا وقتی ترتیب دستورها درست باشه – فصل ۱۷)
- ۲. اشتراک:** دو ایمج که لایه پایه یکسان دارن (مثلاً هر دو `node:22`)، لایه‌ی مشترک فقط یک بار روی دیسکه
- ۳. فقط خواندن بودن:** لایه‌ها تغییر نمی‌کنن – تغییری یعنی لایه جدید

ببین لایه‌های یک ایمج رو:

```
$ docker history postgres:18
IMAGE          CREATED          SIZE
a1b2c3d4e5f6   2 weeks ago     45kB
<missing>      2 weeks ago     7.2MB
<missing>      2 weeks ago     214MB
```

هر خط یک لایه ←
پایه‌های بزرگ در پایین ←

۴.۴ – Pull، به‌روزرسانی و حذف

```
$ docker pull node:22-alpine      # فقط دانلود
$ docker pull postgres:18        # چک آپدیت (لایه‌های جدید را می‌گیرد) = pull دوباره

$ docker rmi nginx:latest         # حذف ایمیج (remove image)
$ docker rmi $(docker images -q)  # (!نگاه کن docker images اول) حذف همه ⚠️
$ docker image prune              # را پاک کن (dangling) ایمیج‌های بدون تگ
$ docker system df                # گزارش فضا: ایمیج/کانتینر/ولیوم/کش چقدر خورده‌اند؟ ★
```

ارور معروف: `image is being used by stopped container` – یعنی اول کانتینرهاش رو `rm` کن. داکر نمی‌ذاره چیزی رو پاک کنی که داره استفاده می‌شه – محافظ خوبی.

۴.۵ – چند ایمیج که دولوپر Next.js می‌شناسه

ایمیج	برای چی
node:22-alpine	اجرا/build اپ‌های Node – سبک‌ترین و رایج‌ترین
postgres:18	دیتابیس دوره‌ات!
redis:7-alpine	کش و صف – در پروژه‌های بزرگ‌تر
nginx:alpine	reverse proxy / سرو کردن فایل‌های استاتیک
hello-world	تست سلامت نصب

💡 **Alpine** چیه؟ یک توزیع مینی‌مال لینوکس (~۵ مگ!) که همه ایمیج‌های رسمی واریانت `alpine` براش دارن – نصف تا یک سوم حجم بقیه. قانون سرانگشتی: برای `production`، `alpine`.

۴.۶ – DIY در Hub: حساب خودت

یک اکانت Docker Hub بساز (رایگان) – از فصل ۱۲ (push و دپلوی) لازمش داری. بعدش `docker login` بزن با همون `username`.

و یک قابلیت جالب: **rate limit** – کاربر ناشناس محدودیت pull داره؛ با لاگین رایگان محدودیتت بالاتره. لاگین کردن عادت خوبی.

✓ جمع‌بندی فصل

• `npm = Docker Hub` ایمیج‌ها؛ `search/pull/images/rmi`

- تگ دقیق بزن (`*` = `latest`) – `postgres:18` در dependency ها
- ایمیج = پشته لایه‌های فقط خواندنی → کش build، اشتراک فضا، تغییرناپذیری
- `docker system df` و `image prune` = مدیریت فضا
- `Alpine` = انتخاب پیش‌فرض production

تمرین فصل ۴

1. `docker images` بزن – چه ایمیج‌هایی داری؟ حجم کل حدوداً چقدر؟
2. `docker pull node:22-alpine` و بعد `docker pull node:22` – حجمشون رو مقایسه کن (چند برابر فرق داره؟).
3. `docker history node:22-alpine` بزن – چند لایه داره؟ بزرگ‌ترین لایه‌اش چیه؟
4. یک کانتینر از `nginx` بساز و متوقفش کن؛ حالا `docker rmi nginx` بزن و ارور رو بخون – بعد کانتینر رو حذف کن و دوباره `rmi`.
5. `docker system df` رو اجرا کن و گزارشش رو تفسیر کن.

▼ نکته تمرین ۲ و ۴

تمرین ۲: `node:22-alpine` حدود ۱۵۰MB و `node:22` حدود ۴۰۰+MB – `alpine` چند برابر سبک‌تر است. تمرین ۴: ارور می‌گوید ایمیج توسط کانتینر متوقف شده استفاده شده: `docker rm <container-id>` بعد `docker rmi` `nginx` – داکر وابستگی‌ها را درست مدیریت می‌کند.

➔ **فصل بعد:** میدون داخل کانتینر – شل بگیر، متغیرها رو ببین، دیباگ کن.

فصل ۵ – داخل کانتینر: exec، متغیرها و دیباگ واقعی

🎯 هدف: یاد بگیری «بروی داخل» کانتینر – شل بگیری، فایل‌ها رو ببینی، متغیرهای محیطی رو چک کنی. این همون مهارتیه که در شرکت، وقتی «کانتینر ارور می‌ده» ازت انتظار دارن.

۵.۱ – ورود به کانتینر: `docker exec -it`

کانتینر = یک لینوکس در حال اجراست – پس می‌شه شل گرفت!

```
$ docker run -d --name web nginx
$ docker exec -it web bash
root@a1b2c3d4:/# «شل داخل کانتینر! حالا «تویی» ←
```

باز کردن فلگ‌ها (دو کلید طلایی): `-i` (interactive) = stdin – باز نگه دار – بتونی تایپ کنی – `-t` (tty) = ترمینال بساز – `prompt` و رنگ و رفتار درست

همیشه با هم: `-it`. دستور `exec` برای اجرای یک دستور یا برنامه جدید درون کانتینری است که در حال حاضر اجرا می‌شود (در اینجا اجرای شل `bash`).

حالا داخلش بازی کن:

```
root@a1b2c3d4:/# ls # فایل‌سیستم لینوکس ایمج nginx
bin dev etc usr var ...

root@a1b2c3d4:/# cat /etc/os-release # چه توزیعی هستیم؟
PRETTY_NAME="Debian GNU/Linux 12 (bookworm)"

root@a1b2c3d4:/# ps aux # خودش nginx پروسه‌های «این کانتینر» – فقط
USER PID %CPU COMMAND
root 1 0.0 nginx: master process

root@a1b2c3d4:/# exit ✓ خروج – کانتینر ادامه می‌دهد اجرا #
```

🔑 نکته‌ی «آهاااا»: `ps` داخل کانتینر فقط پروسه‌های خودش رو می‌بینه (`PID 1 = nginx`). این همون ایزوله‌سازی فصل ۱ است – هر کانتینر دنیای خودشه. و نکته دوم: کانتینر زنده‌ست وقتی پروسه اصلی‌اش زنده‌ست – `exit` از شل، کانتینر رو نمی‌کشه.

۵.۲ – بدون ورود: `exec` برای یک دستور

برای کارهای سریع لازم نیست وارد بشی:

```
$ docker exec web cat /etc/nginx/nginx.conf      # بخوان nginx فایل کانفیگ
$ docker exec web ls /usr/share/nginx/html        # فایل‌های سایت
$ docker exec web env | grep -i nginx             # متغیرهای محیطی‌اش
```

۵.۳ – دیباگ واقعی: چرا کانتینرم بالا نمی‌آید؟

چرخه‌ی استاندارد دیباگ (این رو حفظ کن – در شرکت هر هفته لازمه):

```
$ docker ps -a      # کانتینر هست؟ وضعیتش چیه؟ (۱)
# Exited (1)        # یعنی کرش کرده و پروسه با کد خطای ۱ خارج شده
$ docker logs --tail 50 my-app    # لاگ - ۹۰٪ جواب اینجاست (۲)
$ docker exec -it my-app sh        # داخل شو و دستی تست کن (۳)
# (docker run -it --entrypoint sh node:22-alpine) کانتینر مرده؟ از روی ایمیجش یک کانتینر موقت بزن
$ docker inspect my-app           # مانت‌ها، env، همه جزئیات: پورته‌ها (۴)
```

⚠ نکته‌ی sh به جای bash : ایمیج‌های alpine برای سبکی، bash ندارند – sh دارن. اگه `exec -it x bash` ارور داد، alpine اس؛ sh بزن.

۵.۴ – `docker inspect` : شناسنامه کانتینر

خروجی JSON بزرگه – با ابزارهای شل فیلترش کن:

```
$ docker inspect my-web | grep -A 5 "PortBindings"
$ docker inspect --format '{{.Config.Image}}' my-web    # nginx
$ docker inspect --format '{{.State.Status}}' my-web    # running
```

(چیز مفیدی دیدی؟ `--format` با `template` – مثل `template literal` ها.)

۵.۵ – کپی فایل بین میزبان و کانتینر

```
$ docker cp my-web:/usr/share/nginx/html/index.html ./      # از کانتینر به سیستم
$ docker cp ./new-page.html my-web:/usr/share/nginx/html/    # از سیستم به کانتینر
```

(مشابه انتقال فایل در لینوکس یا شبکه.) برای دیباگ/استخراج لاگ/گرفتن فایل کانفیگ عالیه. ⚠ ولی توجه: این تغییر فقط در این کانتینر است – کانتینر rm شه می‌پره. راه درست ماندگاری = `volume` (فصل ۹) و راه درست تغییر ایمیج = `Dockerfile` (فصل ۶).

۵.۶ – متغیرهای محیطی از نگاه داخل

امتحان کن (از فصل ۳):

```
$ docker run -d --name db -e POSTGRES_PASSWORD=secret -e APP_MODE=dev postgres:18

$ docker exec db env | grep -E "POSTGRES|APP_MODE"
POSTGRES_PASSWORD=secret
APP_MODE=dev
```

این دقیقاً همان مقادیری است که برنامه از طریق `process.env` دریافت می‌کند – جریان کانفیگ: `-e` یا فایل `env` → متغیر محیطی کانتینر → برنامه داخلش می‌خونه. (در فصل ۸ همینو با `compose` حرفه‌ای‌تر می‌کنیم.)

جمع‌بندی فصل ✓

- `docker exec -it <name> bash` = ورود؛ `sh` برای ایمپج‌های `alpine`
- داخل کانتینر: دنیای خودش – `ps` فقط پروسه‌های خودش را می‌بیند
- چرخه دیباگ: `inspect` → `exec` → `logs` → `ps -a`
- `docker cp` = انتقال فایل بین میزبان و کانتینر (موقتی – دائمی‌اش `volume/Dockerfile`)
- کانفیگ از بیرون = `env` → `-e` داخل کانتینر → برنامه می‌خواند

تمرین فصل ۵ 📝

1. `nginx` اجرا کن، واردش شو (`exec -it ... bash`)، نسخه `Debian` را بخوان، `nginx -v` را اجرا کن و خارج شو.
2. با `docker exec` (بدون ورود) فایل `/etc/nginx/nginx.conf` را بخوان و با `grep` عبارت `server {` را در آن پیدا کن.
3. یک کانتینر با دو متغیر محیطی بساز (`-e GREETING="سلام" -e ENV_NAME=test`) و از داخلش `env` بزن و هر دو را ببین.
4. چالش دیباگ ★: یک کانتینر بنویس که کرش کنه: `docker run --name crasher alpine sh -c "echo boom; exit 1"`
5. یک فایل از کانتینر `nginx` به سیستم کپی کن و محتوایش را `cat` کن.

▼ جواب تمرین ۴

```
docker run --name crasher alpine sh -c "echo boom; exit 1"
docker ps -a --filter name=crasher      # کد خروج ۱ ← Exited (1) وضعیت
docker logs crasher                    # خروجی: boom
docker rm crasher
```

➔ فصل بعد: ساخت اولین `Dockerfile` و بسته‌بندی یک پروژه وب!

فصل ۶ – اولین Dockerfile: بسته‌بندی اپ Node خودت

🎯 هدف: تا حالا ایمیج‌های دیگران رو اجرا کردی؛ حالا ایمیج خودت رو می‌سازی. یک اپ Node ساده رو قدم‌به‌قدم در Dockerfile بسته‌بندی می‌کنیم – خط به خط، با چرایی هر دستور.

۶.۱ – Dockerfile چیه؟

Dockerfile = دستور پخت ایمیج تو. یک فایل متنی که می‌گه: از چه پایه‌ای شروع کن، چه فایل‌هایی بیار، چه نصب‌هایی کن، و در آخر چه دستوری اجرا شه. داکر از رویش ایمیج می‌سازه:



(مثل فرق recipe و غذای آماده: Dockerfile دستوره، ایمیج غذای بسته‌بندی‌شده‌ی تکرارپذیره.)

۶.۲ – اپ نمونه: یک API مینی‌مال

یک پوشه بساز: `~/docker-playground/myapi` و دو فایل:

`: package.json`

```
{
  "name": "myapi",
  "version": "1.0.0",
  "scripts": {
    "start": "node server.js"
  },
  "dependencies": {
    "express": "^5.0.0"
  }
}
```

`: server.js`

```
const express = require("express");
const app = express();

app.get("/", (req, res) => {
  res.json({ message: "سلام از داخل کانتینر 🌐", host: require("os").hostname() });
});

app.listen(3000, () => console.log("API روی پورت 3000 بالا آمد 🚀"));
```

۶.۳ – اولین Dockerfile (خط به خط)

فایل `Dockerfile` (بدون پسوند) در همین پوشه:

```
FROM node:22-alpine

WORKDIR /app

COPY package.json ./

RUN npm install

COPY . .

EXPOSE 3000

CMD ["node", "server.js"]
```

خط به خط:

دستور	کار	چرا
<code>FROM node:22-alpine</code>	ایمیج پایه – از اینجا شروع کن	Node 22 + لینوکس مینیمال؛ ما از صفر OS نمی‌سازیم!
<code>WORKDIR /app</code>	پوشه کاری داخل کانتینر	مثل <code>cd /app</code> – همه دستورات بعدی اونجا
<code>COPY package.json ./</code>	فقط فایل‌های وابستگی رو بیار	🌟 راز کش – بخش ۶.۵
<code>RUN npm install</code>	حین <i>build</i> اجرا کن (نصب وابستگی‌ها)	نتیجه در لایه ایمیج ذخیره می‌شه
<code>COPY . .</code>	حالا کل سورس رو بیار	آخر از همه تا با هر تغییر کد، نصب مجدد نشه!
<code>EXPOSE 3000</code>	مستندسازی: «این اپ روی ۳۰۰۰ گوش می‌ده»	⚠️ پورت رو باز نمی‌کنه! فقط متادیتا – باز کردن با <code>-p</code> موقع <code>run</code>
<code>CMD ["node", "server.js"]</code>	موقع <i>run</i> اجرا کن	پروسه اصلی کانتینر – هر کانتینر یک CMD

🔑 مهم‌ترین تفاوت: **RUN** موقع **build** (در ایمپج ثبت می‌شه)، **CMD** موقع **run** (پروسه اصلی). فرقشون مثل «نصب برنامه» و «اجرا کردن برنامه» است.

۶.۴ – **run** و **build**: لحظه حقیقت

در Git Bash، داخل پوشه **myapi**:

```
$ docker build -t myapi:1.0 .
#           ↑ مسیر ↑ اسم:تگ خودم build context (نقطه = همین پوشه)
[+] Building 23.4s (10/10) FINISHED
=> [4/6] RUN npm install          نصب وابستگی‌ها
=> [6/6] CMD ["node","server.js"] لایه‌ها ساخته شدند
> naming to docker.io/library/myapi:1.0

$ docker images | grep myapi
myapi   1.0    a1b2c3d4e5f6   10 seconds ago   132MB

$ docker run -d -p 3000:3000 --name api myapi:1.0

$ curl http://localhost:3000
{"message":"! سلام از داخل کانتینر!", "host":"a1b2c3d4e5f6"}
```

اپ **خودت** الان توی کانتینره: Node نصب، وابستگی‌ها نصب، و روی هر سیستمی همون‌طور اجرا می‌شه. **host** در پاسخ ID = کانتینره (hostname داخل ایزوله‌سازی).

۶.۵ – کش لایه‌ها: چرا ترتیب دستورها مهمه ★

سورس رو عوض کن (پیام API رو) و دوباره **build** بزن:

```
$ docker build -t myapi:1.0 .
=> [3/6] COPY package.json ./      ✓ CACHED
=> [4/6] RUN npm install           ✓ CACHED ← ! نصب نشد دوباره
=> [5/6] COPY . .                  (جدید - چون سورس عوض شد)
```

چون **package.json** تغییر نکرده، لایه‌های نصب از کش اومدن – **build** از ۲۳ ثانیه به ۱ ثانیه. **قانون طلایی Dockerfile**: چیزهای به‌ندرت‌تغییرشونده بالا، چیزهای پرتغییرشونده (سورس) پایین. همون ایمپج که نوشتی درست همین کارو کرد.

۶.۶ – **dockerignore**: مثل **gitignore**

قبل از **build**، داکر کل پوشه رو به **build context** می‌فرسته – شامل **node_modules** (صد مگ!) و **.git**. فایل **.dockerignore** (کنار Dockerfile):

```
node_modules
.git
.env
*.log
Dockerfile
.dockerignore
```

بدون این، `COPY` همه چیز رو داخل ایمج می‌ریزه (شامل `node_modules` لوکال که با مک/ویندوز ناسازگاره!) و `.env` هم لو می‌ره. از همین امروز: هر پروژه‌ی داکری `.dockerignore` داره. همیشه.

✓ جمع‌بندی فصل

- Dockerfile = دستور پخت؛ `docker build -t name:tag .` → ایمج؛ `run` → کانتینر
- هفت دستور پایه: FROM / WORKDIR / COPY / RUN / EXPOSE / CMD (+ ENV از فصل قبل)
- RUN = حین build؛ CMD = پروسه حین run؛ EXPOSE فقط مستند
- ترتیب = کش: وابستگی‌ها اول، سورس آخر
- `.dockerignore` همیشه: `node_modules` و `env` و `git`.

📝 تمرین فصل ۶

1. اپ `myapi` را دقیقاً مثل فصل بساز، `build` و `run` کن و با `curl` تست کن.
2. پیام JSON را عوض کن (مثلاً اسمت را اضافه کن) و دوباره `build` — ببین کدام لایه‌ها CACHED شدند.
3. `EXPOSE 3000` را از Dockerfile حذف کن، `rebuild` و بدون فلگ `-p` اجرا کن — `curl` کار می‌کند؟ چرا؟ (بعد برگردون!)
4. `.dockerignore` بساز و با `docker build` دوباره — تفاوت سرعت/اندازه را با `docker images` ببین.
5. چالش: یک Dockerfile برای یک فایل اسکریپت ساده بنویس (فقط `node script.js` که یک `console.log` دارد) — از ایمجش `node:22-alpine` و بدون `npm install`.

▼ جواب تمرین ۳ و ۵

تمرین ۳: نه — `curl fail` می‌شود (connection refused). `EXPOSE` پورت را باز نمی‌کند؛ فقط مستندسازی است. کانتینر اجرا شده ولی از دنیای بیرون قابل دسترس نیست مگر با `-p`. (داخل کانتینر با `exec` و `curl/wget` می‌شد دیدش!) تمرین ۵:

```
FROM node:22-alpine
WORKDIR /app
COPY script.js .
CMD ["node", "script.js"]
```

→ فصل بعد: ایمپج را حرفه‌ای کن – multi-stage و کاهش چشمگیر حجم.

فصل ۷ – ایمیج تمیز: multi-stage و کاهش حجم

🎯 هدف: Dockerfile فصل ۶ کار می‌کنه ولی «مبتدیان» است. یاد می‌گیری چرا ایمیج‌های Node معمولی چاقان، و با multi-stage build اون‌ها رو به ایمیج‌های تمیز production تبدیل کنی – همون چیزی که در Dockerfile های شرکت‌ها می‌بینی.

۷.۱ – مشکل: ایمیج من چرا اینقدر چاقه؟

ایمیج myapi فصل قبل ۱۳۲MB بود برای یک API ده‌خطی! چرا؟ چون همه‌چیز تو اونه:

چی داخلشه	حجم	ولی برای اجرا لازمه؟
لایه پایه Node + alpine	~۱۲۰MB	بله ✓
npm cache (باگ یا نه، npm کش می‌سازه)	ده‌ها MB	نه ✗
node_modules شامل devDependencies	عظیم	✗ نه – فقط برای build/test
کد سورس کامل	کم	⚠ در production فقط خروجی build لازمه

و برای Next.js بدتر: در build به همه devDependencies نیاز داری، ولی برای اجرا فقط خروجی next + وابستگی‌های production لازمه.

۷.۲ – راه‌حل: Multi-Stage Build ★

فکرش ساده‌ست: دو مرحله – یکی برای build (می‌تونه چاق باشه)، یکی برای اجرا (فقط خروجی رو می‌بره).

```
# — (ابزارهای سنگین اینجاست) مرحله ۱ —
FROM node:22-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

# — (فقط خروجی) مرحله ۲ —
FROM node:22-alpine
WORKDIR /app

COPY package*.json ./
RUN npm install --omit=dev && npm cache clean --force

COPY --from=builder /app/dist ./dist

EXPOSE 3000
CMD ["node", "dist/server.js"]
```

خط به خط چیزهای جدید:

دستور	کار
AS builder	اسم‌گذاری مرحله اول
npm install --omit=dev	فقط وابستگی‌های production (بدون devDependencies مثل TypeScript و eslint)
npm cache clean --force	حذف کش npm از همون لایه — جلوگیری از قچاق شدن ایمج
COPY --from=builder /app/dist ./dist	★ جادو: فقط پوشه خروجی رو از مرحله build بکش بیرون

نتیجه: مرحله builder (با eslint، typescript، کش‌ها و...) **دور انداخته می‌شه** — ایمج نهایی فقط runtime است. حجم معمولاً از صدها مگ به ده‌ها مگ می‌رسه (۳-۵ برابر کوچک‌تر).

🔑 چرا امن‌تر هم هست؟ چون سورس کامل، devDependencies و ابزارهای build تو ایمج نهایی نیستن — سطح حمله کمتر. (این جمله رو در مصاحبه بگو، امتیاز می‌گیری 😊)

۷.۳ — Next.js واقعی (الگوی رسمی)

الگوی multi-stage برای Next.js (که خود Next.js در مستنداتش هم می‌ده — با `output: "standalone"` در `next.config`):

```
FROM node:22-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM node:22-alpine
WORKDIR /app
ENV NODE_ENV=production
COPY --from=builder /app/public ./public
COPY --from=builder /app/.next/standalone ./
COPY --from=builder /app/.next/static ./next/static

EXPOSE 3000
CMD ["node", "server.js"]
```

- `npm ci` به جای `npm install` : نصب دقیق از `package-lock.json` – سریع تر و تکرارپذیر (مثل قفل شدن نسخه‌ها).

۷.۴ – چند قانون طلایی حجم

قانون	چرا
پایه alpine/slim	پایه ۱۲۰MB → ۵۰MB
<code>RUN</code> های مرتبط رو با <code>&&</code> یکی کن	هر <code>RUN</code> یک لایه – لایه حذف شده فضا آزاد نمی‌کنه!
<code>npm cache clean</code> در همین <code>RUN</code>	فایل اضافه شده در لایه قبلی، در لایه بعد پاک نمی‌شه
<code>npm ci --omit=dev</code> در آخرین مرحله	devDependencies نیز
چند stage در صورت نیاز	فایل‌های build از ایمج نهایی بیرون می‌مونن

نمونه‌ی قانون «یکی‌کردن `RUN`»:

```
# ❌ در لایه قبلی می‌ماند npm کش:
RUN npm install
RUN npm cache clean --force

# ✅ همه در یک لایه – پاک‌سازی داخل همان لایه:
RUN npm install --omit=dev && npm cache clean --force
```

۷.۵ – measure: چطوری بفهمم چقدر لاغر شد؟

```
$ docker build -t myapi:2.0 .
$ docker images myapi
myapi    1.0    ...    132MB    ← نسخه تکمرله‌ای
myapi    2.0    ...    96MB     ← multi-stage + omit=dev بعد از

# جزئیات لایه به لایه:
$ docker history myapi:2.0
```

و ابزار تحلیل حرفه‌ای: (github.com/wagoodman/dive) `dive` – داخل ایمج رو لایه‌به‌لایه نشون می‌ده و می‌گه چه چیزی هدر رفته. (فعلاً فقط اسمش رو بدونی کافیه.)

✓ جمع‌بندی فصل

- ایمج چاق = ابزارهای build + devDeps + کش‌ها داخل ایمج نهایی
- **Multi-stage**: مرحله builder (چاق، دور انداخته می‌شود) → مرحله final (فقط خروجی + runtime)
- `COPY --from=builder` = انتقال فقط خروجی؛ `npm ci --omit=dev` = فقط production deps
- پاک‌سازی باید داخل همان RUN باشد (لایه‌ها فقط خواندن‌اند)
- حجم کمتر = دیپلوی سریع‌تر + سطح حمله کمتر

📄 تمرین فصل ۷

1. Dockerfile فصل ۶ را به multi-stage تبدیل کن (حتی اگر اپ build ندارد – یک مرحله deps و یک مرحله runtime با `omit=dev` و فرق حجم را با `docker images` ببین).
2. در Dockerfile، دو RUN پشت هم بنویس (نصب + پاک‌سازی جدا) و بعد `docker history` بزن – لایه پاک‌سازی را پیدا کن. چرا SIZE اش 0 یا کم است ولی فضا آزاد نشده؟ (جواب: فایل‌های حذف‌شده در لایه قبلی زنده‌ان!)
3. `npm ci` رو جایگزین `npm install` کن – اگه package-lock نداری، اول یک بار `npm install` لوکال بزن تا ساخته شه.
4. چالش: Dockerfile مربوط به Next.js در بخش ۷.۳ را روی یک پروژه Next.js امتحان کن (`output: 'standalone'` را به `next.config.js` اضافه کن) و تغییر حجم ایمج را بررسی کن.

▼ جواب تمرین ۲

لایه‌ها فقط خواندنی و انباشته‌اند: اگر در لایه ۱ فایل ۱۰۰MB بسازی و در لایه ۲ حذفش کنی، فایل در لایه ۱ هنوز هست – ایمج نهایی هر دو لایه را حمل می‌کند (فقط دیده نمی‌شود). پاک‌سازی باید در همان لایه‌ای انجام شود که فایل ساخته شده (با `&&`).

[→](#) **فصل بعد:** وقتی چند کانتینر لازم داری (اپ + دیتابیس + ...) – Docker Compose!

فصل ۸ – Docker Compose: کل استک با یک دستور ★

🎯 هدف: پروژه واقعی یک کانتینر نیست – اپ + دیتابیس + شاید redis. یاد می‌گیری همه رو با یک فایل YAML تعریف و با یک دستور بالا بیاری: `docker compose up`. پرکاربردترین ابزار روزمره‌ی دولوپرها.

۸.۱ – مسئله: ده دستور برای هر بار بالا آوردن استک

بدون compose، استک واقعی پروژه یعنی:

```
docker run -d --name db -e POSTGRES_PASSWORD=1234 -p 5432:5432 postgres:18
docker run -d --name redis -p 6379:6379 redis:7-alpine
docker run -d --name api -p 3000:3000 --link db... myapi:1.0 # و.اتصالات و ترتیب‌ها
```

تکرار دستی این دستورات در هر بار راه‌اندازی، توسط هر عضو تیم یا در محیط‌های مختلف، بسیار خطاپذیر است و مدیریت فلگ‌ها و پورت‌ها را دشوار می‌کند.

جواب Compose: همه رو در یک فایل `compose.yaml` تعریف کن (نسخه‌پذیر در !git) و:

```
docker compose up -d # جهان داکر npm install + start همه با هم – همون
```

۸.۲ – اولین compose.yaml

در پوشه پروژه (کنار Dockerfile)، فایل `compose.yaml`:

```
services:
  api:
    build: . # همین پوشه بساز Dockerfile از
    ports:
      - "3000:3000"
    environment:
      - NODE_ENV=production
    depends_on:
      - db
  db:
    image: postgres:18
    environment:
      POSTGRES_PASSWORD: mysecretpass
      POSTGRES_DB: shop
    ports:
      - "5432:5432"
```

آناتومی:

کلید	معنی
<code>services:</code>	هر سرویس = یک کانتینر از یک ایمج
<code>build: .</code>	از Dockerfile این پوشه build کن (به جای image آماده)
<code>image: postgres:18</code>	یا از ایمج آماده
<code>ports: - "3000:3000"</code>	همون <code>-p</code> (میزبان:کانتینر)
<code>environment:</code>	همون <code>-e</code> ها – و چون YAML است، خواناتر
<code>depends_on:</code>	ترتیب: اول db بالا بیاد، بعد api

۸.۳ – سه دستور طلایی

```
$ docker compose up -d      # کافیه – (در صورت نیاز build) بالا بیاور
$ docker compose ps         # وضعیت سرویس‌ها
$ docker compose logs -f api # لاگ یک سرویس (یا همه: بدون اسم)
$ docker compose down       # (می‌مانند volumes ایمج‌ها و) توقف + حذف کانتینرها و شبکه
```

حتی `build` هم لازم نیست جدا بزنی – `up` خودش تشخیص می‌ده. و برای «کد عوض شد، دوباره بساز»:

```
$ docker compose up -d --build
```

💡 سرویس‌ها با اسمشون صدا زده می‌شن: `docker compose exec api sh` ، `docker compose restart db` – همون دستورهای فصل ۵، فقط به جای اسم کانتینر، اسم سرویس.

۸.۴ – پروژه نمونه: استک واقعی (api + db)

همون اپ `myapi` فصل ۶ – ولی این‌بار با دیتابیس. اول کد رو به Postgres وصل کنیم (`server.js` ساده‌شده):

```
const express = require("express");
const { Pool } = require("pg");
const app = express();

const pool = new Pool({
  connectionString: process.env.DATABASE_URL, // کانتینر از environment!
});

app.get("/", async (req, res) => {
  const { rows } = await pool.query("SELECT version()");
  res.json({ db: rows[0].version.slice(0, 30) });
});

app.listen(3000);
```

(توجه: پیش از بیلد، پکیج `pg` را با دستور `npm i pg` به پروژه اضافه کرده و در `package.json` قرار دهید.)
و `compose.yaml` نهایی:

```
services:
  api:
    build: .
    ports:
      - "3000:3000"
    environment:
      DATABASE_URL: postgres://postgres:mysecretpass@db:5432/shop # ← host = اسم سرویس!
    depends_on:
      - db

  db:
    image: postgres:18
    environment:
      POSTGRES_PASSWORD: mysecretpass
      POSTGRES_DB: shop
    volumes:
      - pgdata:/var/lib/postgresql/data # داده ماندگار (فصل ۱۹)

volumes:
  pgdata:
```

اجرا و تست:

```
$ docker compose up -d --build
$ curl http://localhost:3000
{"db": "PostgreSQL 18.4 (Debian 12...)"}

```

API تو از داخل یک کانتینر به دیتابیس داخل کانتینر دیگه وصل شد. 🇮🇷 (راز `@db:5432` – فصل ۱۰)

۸.۵ – `env.` در `compose` (همون الگوی همیشگی) ★

پسورد داخل YAML که توی git می‌ره؟ نه! الگوی استاندارد:

`.env` (کنار `compose.yaml`، در `.gitignore`):

```
POSTGRES_PASSWORD=mysecretpass
```

: `compose.yaml`

```
db:
  image: postgres:18
  environment:
    POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
```

Compose خودش فایل `.env` کنار خودش رو می‌خونه و جایگزین می‌کنه – مقادیر در دو لایه تزریق می‌شوند: `env` فایل `env` → YAML کانتینر → برنامه. (توصیه حرفه‌ای: همیشه یک فایل `.env.example` بدون مقادیر حساس برای مخزن گیت ایجاد کنید تا سایر توسعه‌دهندگان از ساختار متغیرها مطلع شوند.)

```
$ docker compose up -d --build db          # فقط یک سرویس
$ docker compose exec db psql -U postgres  # اجرای دستورات مدیریت دیتابیس در کانتینر
$ docker compose restart api
$ docker compose down -v                   # داده‌ها می‌روند - volumes با حذف ⚠️
$ docker compose config                    # برای چک - (جایگزین‌شده env با) YAML خروجی نهایی
```

✓ جمع‌بندی فصل

- استک چندکانتینره = `compose.yaml` در `docker compose up -d` + `git`
- `run` = `ports/environment/depends_on` ؛ `build:` vs `image:` = معادل فلگ‌های
- `down` کانتینرها را می‌برد؛ با `-v` داده هم می‌رود (خطرناک!)
- راز اتصال: اسم سرویس = `@db:5432` (hostname) – فصل ۱۰ کامل
- رازها با `.env` کنار `compose` – هرگز در YAML خالی

📄 تمرین فصل ۸

۱. استک `api + db` این فصل را کامل بساز و بالا بیا؛ `curl` بزن و پاسخ PostgreSQL را ببین.
۲. `docker compose ps` و `docker compose logs -f api` را تست کن.
۳. با `docker compose exec db psql -U postgres -d shop` برو داخل دیتابیس و `\l` بزن.
۴. `compose` را `down` کن و دوباره `curl` – `up` دوباره جواب می‌دهد؟ (بله! چون ایمیج و volume باقی‌اند).
۵. پسورد را به `.env` منتقل کن و با `docker compose config` صحت جایگزینی را چک کن.

▼ نکته تمرین ۲

`logs -f api` فقط لاگ سرویس `api` را زنده نشان می‌دهد؛ بدون اسم سرویس، همه سرویس‌ها قاطی می‌آیند (با پیشوند اسم سرویس) – برای دیدن «همه‌چیز با هم» موقع `startup` عالی است.

➔ **فصل بعد:** volume ها – داده‌ای که با `down` نمی‌میرد و توسعه‌ی زنده با `bind mount`.

فصل ۹ – Volumes: داده ماندگار و توسعه‌ی زنده

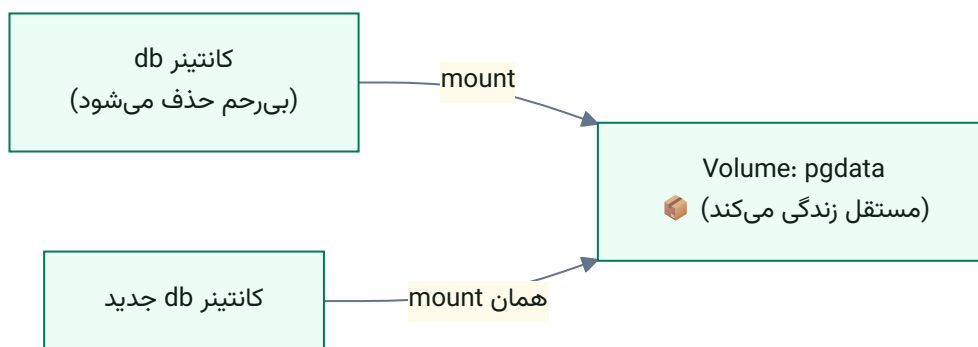
🎯 هدف: دو مشکل واقعی رو حل می‌کنیم: (۱) داده‌ی دیتابیس با حذف کانتینر می‌میره – named volume حلشه؛ (۲) در توسعه، هر تغییر کد نیاز به build دوباره داره – bind mount حلشه.

۹.۱ – داده‌ها و ماهیت یکبارمصرف کانتینرها (Disposable / Ephemeral)

اصل بنیادین در داکر: کانتینرها ماهیتاً موقتی و یکبارمصرف هستند و با حذف کانتینر، داده‌های نوشته‌شده درون آن از بین می‌رود. این موضوع را در عمل بررسی کنیم:

```
$ docker run -d --name db1 -e POSTGRES_PASSWORD=1 postgres:18
# چند رکورد درش ثبت کن.
$ docker rm -f db1 # حذف
$ docker run -d --name db2 -e POSTGRES_PASSWORD=1 postgres:18
# دیتابیس خالی! همه داده‌ها رفت.
```

داده‌ها داخل لایه‌ی نوشتنی کانتینر بودن که با `rm` حذف می‌شه. جواب داکر: **Volume** – یک فضای مدیریت‌شده‌ی بیرون کانتینر که بهش وصل می‌شه:



۹.۲ – Named Volume: داده‌ای که از death می‌ترسه نه

```
$ docker volume ls # لیست volumes
$ docker volume create pgdata # ساخت دستی (معمولاً لازم نیست)
$ docker volume rm pgdata # حذف (⚠ داده برای همیشه می‌رود)
$ docker volume inspect pgdata # کجا روی دیسک ذخیره شده؟
```

و استفاده در `run`:

```
$ docker run -d --name db \
-e POSTGRES_PASSWORD=1 \
-v pgdata:/var/lib/postgresql/data \
postgres:18
#      ↑ اسم volume      ↑ مسیر داخل کانتینر
```

فرم **مسیر کانتینر: volume اسم -v : -v** — یعنی «این مسیر داخل کانتینر را به این volume وصل کن». حالا **rm** کانتینر، داده رو نمی‌کشد: کانتینر جدید رو به همون volume وصل کن، دیتا برگرده.

توی compose (همون که فصل ۸ دیدی):

```
services:
  db:
    image: postgres:18
    volumes:
      - pgdata:/var/lib/postgresql/data

volumes:
  pgdata:      # تعریف سطح بالای فایل ←
```

۹.۳ — Bind Mount: اتصال پوشه‌ای از سیستم میزبان به داخل کانتینر

فرقش با named volume: به جای فضای مدیریتی داکر، یک پوشه واقعی از سیستم میزبان رو وصل می‌کنی. فرم تشخیصی: چون به جای یک اسم ساده، مسیر فایل سیستم درج می‌شود:

```
$ docker run -d --name web -p 8080:80 \
-v "$(pwd)/site":/usr/share/nginx/html \
nginx
```

حالا فایل‌های پوشه‌ی **site** روی سیستم میزبان = محتوای سایت nginx. امتحانش:

```
$ echo "<h1>سلام از وب‌سرور داکر</h1>" > ./site/index.html
$ curl http://localhost:8080
<h1>سلام از وب‌سرور داکر</h1>      # بدون restart، بدون rebuild
```

۹.۴ — در compose: bind mount برای توسعه ⭐

این الگو = قلب توسعه‌ی روزمره با داکر:

```
services:
  api:
    build: .
    volumes:
      - ./src:/app/src      # پوشه سورس کد → داخل کانتینر
    environment:
      NODE_ENV: development
```

با **node --watch server.js** (یا nodemon) به عنوان CMD در توسعه: فایل رو ذخیره می‌کنی → داخل کانتینر همون لحظه تغییر کرده → سرور reload می‌شه. تجربه‌ی **hot-reload** معمولی، ولی داخل کانتینر!

ویژگی	Named Volume	Bind Mount
فرم -v	اسم: مسیر	مسیر/سیستم: مسیر
ماهیت	فضای ذخیره‌سازی تحت مدیریت داکر	پوشه واقعی روی سیستم‌عامل میزبان
بهترین کاربرد	داده‌های پایدار (دیتابیس)	محیط توسعه: سینک زنده سورس کد
وابستگی محیطی	کاملاً مستقل و قابل حمل	وابسته به ساختار مسیر سیستم میزبان

۹.۵ – بکاپ: چون «ماندگار» = «باید بکاپش رو داشته باشم»

```
# mount کانتینر موقت برای volume خروجی گرفتن از محتوای
$ docker run --rm -v pgdata:/data -v "$PWD":/backup alpine \
  tar czf /backup/pgdata-backup.tar.gz /data

# بازگرداندن:
$ docker run --rm -v pgdata:/data -v "$PWD":/backup alpine \
  tar xzf /backup/pgdata-backup.tar.gz -C /
```

(الگوی رایج: کانتینر alpine موقتی که به volume و پوشه بکاپ mount می‌شه — `rm` یعنی بعدش خودش می‌ره.)

۹.۶ – قانون‌های ماندگاری

۱. دیتابیس بدون volume = بمب ساعتی. همیشه named volume
۲. سورس در توسعه = bind mount؛ در production = داخل ایمج (فصل ۷ – COPY)
۳. `docker compose down` با volume ها کاری ندارد؛ `-v` = حذف (⚠️ سوال بپرس قبل از زدن!)
۴. `docker system df` حجم volumes را نشان می‌دهد؛ `docker volume prune` = پاکسازی volume های بی‌صاحب

✓ جمع‌بندی فصل

- کانتینر حذف = داده لایه‌اش حذف؛ volume = مستقل از کانتینر زندگی می‌کند
- اسم: مسیر -v (Named volume) برای دیتای production؛ مسیر: مسیر / -v (Bind mount) برای توسعه
- hot-reload = bind mount واقعی داخل کانتینر با `nodemon` / `watch`
- بکاپ با کانتینر موقت alpine؛ `down` امن، `-v down` خطرناک

تمرین فصل ۹

1. Postgres با named volume بالا بیاور (`pgdata`)، با `psql` یک جدول بساز و چند رکورد بزن؛ کانتینر را `rm` کن؛ دوباره با همان volume بالا بیاور — داده‌ها هست؟
2. `bind mount` روی `nginx`: یک پوشه با سه فایل `html` بساز و `mount` کن — بین مرورگر و ادیتور زندگی کن (تغییر → `refresh`).
3. بکاپ volume تمرین ۱ را با الگوی `alpine` بگیر و فایل `tar` را ببین.
4. در `compose` پروژه فصل ۸، به `api` یک `bind mount` برای `src` اضافه کن و `NODE_ENV=development` بگذار.
5. `docker system df` بزن — volumes چقدر گرفته‌اند؟

▼ راهنمای تمرین ۱

```
docker run -d --name pg -e POSTGRES_PASSWORD=1 -v pgdata:/var/lib/postgresql/data postgres:18
psql "postgresql://postgres:1@localhost:5432/postgres" -c "CREATE TABLE t1(x int); INSERT INTO
docker rm -f pg
docker run -d --name pg -e POSTGRES_PASSWORD=1 -v pgdata:/var/lib/postgresql/data postgres:18
psql "postgresql://postgres:1@localhost:5432/postgres" -c "SELECT * FROM t1;" # هنوز هست ←
```

→ فصل بعد: شبکه‌ی کانتینرها — چرا در `DATABASE_URL` نوشتیم `@db:5432` و کار کرد؟!

فصل ۱۰ – شبکه‌ی کانتینرها: چرا @db:5432 کار می‌کرد؟

🎯 هدف: معمای فصل ۸ رو حل می‌کنیم: چطور API داخل یک کانتینر با hostname ی به اسم db به کانتینر دیگه‌ای وصل شد؟ پایان فصل، شبکه‌ی داکر رو مثل localhost می‌فهمی.

۱۰.۱ – سه سناریوی اتصال (و ارور معروف اولی)

سناریو ۱: برنامه‌ی تو (روی میزبان) می‌خواد به کانتینر وصل شه – مثل psql لوکالت به Postgres کانتینری:

```
DATABASE_URL="postgresql://postgres:pass@localhost:5432/shop"
↑ localhost + پورت مپ‌شده (-p 5432:5432)
```

کار می‌کنه چون -p پورت کانتینر رو به localhost میزبان وصل کرد (فصل ۳).

سناریو ۲: دو کانتینر می‌خوان به هم وصل‌شن – و اینجا localhost کار نمی‌کنه!

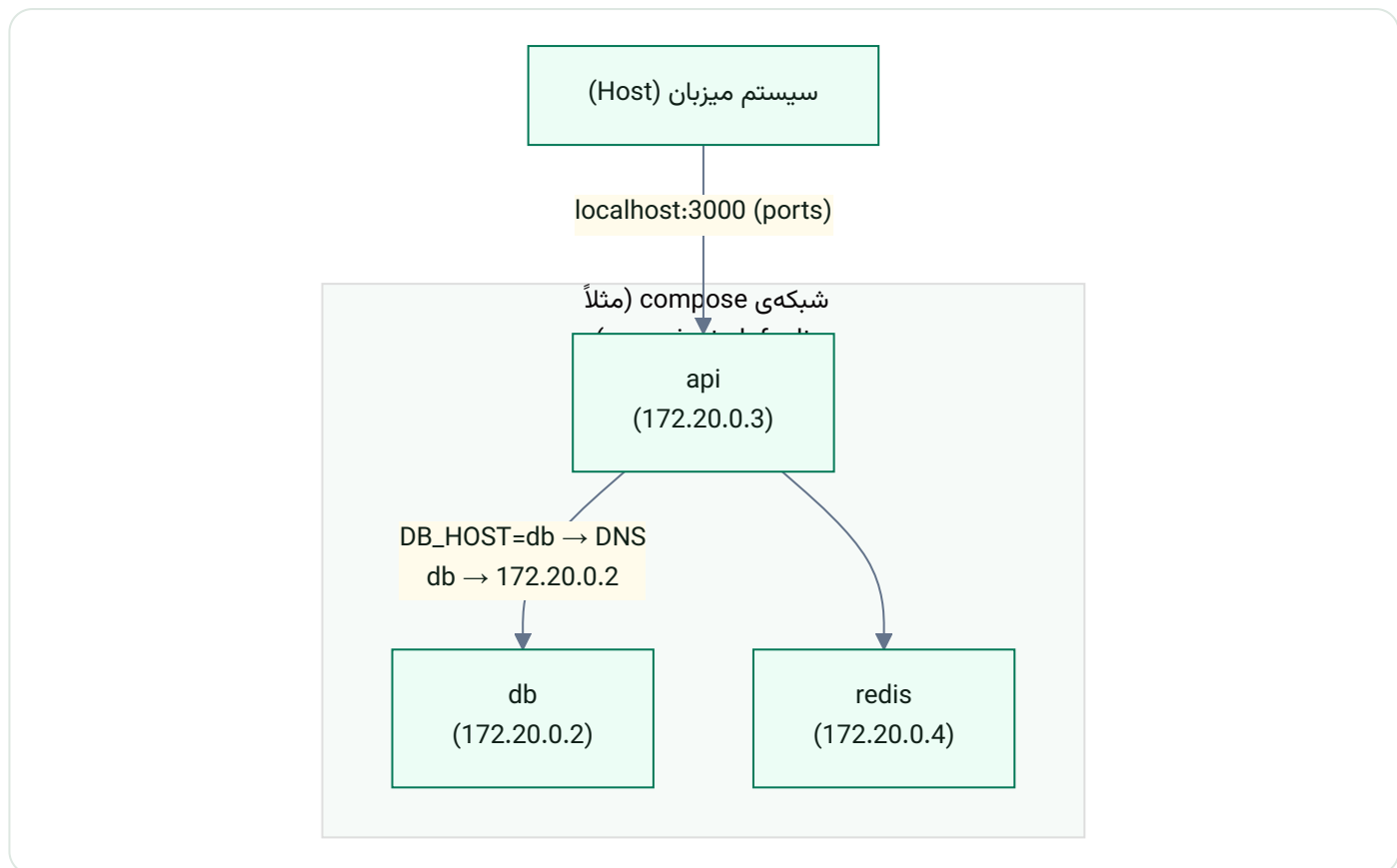
```
# داخل کانتینر api:
DATABASE_URL="...@localhost:5432/..." # ❌ ECONNREFUSED
```

چرا؟ چون هر کانتینر شبکه‌ی خودش رو داره – localhost داخل کانتینر api یعنی «خود کانتینر api»، نه کانتینر db! این پرتکرارترین گیجی دولوپرهای تازه‌کار داکره.

سناریو ۳ (جواب): اسم سرویس به جای localhost – واینکه در compose نوشتیم @db:5432 و کار کرد، چون...

۱۰.۲ – جادوی DNS: Compose داخلی + شبکه‌ی مشترک

- وقتی docker compose up می‌زنی، داکر: 1. یک شبکه‌ی خصوصی برای پروژه می‌سازه و همه سرویس‌ها رو توش می‌ذاره
2. به هر کانتینر DNS داخلی می‌ده: اسم سرویس → آی‌پی کانتینر



یعنی `db` مثل یک اسم دامنه‌ی واقعی کار می‌کند — `postgres://...@db:5432` یعنی «کانتنری که سرویس `db` اسم دارد، پورت ۵۴۳۲ش». بدون آی‌پی، بدون مدیریت دستی.

ببینش:

```
$ docker network ls # (یکی برای هر پروژه) شبکه‌ها
$ docker network inspect myproject_default # چه کسانی عضو + آی‌پی‌ها
```

💡 نکته مهم درباره `depends_on` : این گزینه فقط ترتیب استارت کانتنرها را مشخص می‌کند، نه اینکه کانتنر مقصد لزوماً درخواست‌ها را آماده دریافت کرده باشد. به همین دلیل دیتابیس‌هایی مثل Postgres چند ثانیه زمان می‌برند تا آماده اتصال شوند و برنامه‌ها بهتر است قابلیت `retry` خودکار داشته باشند یا از مکانیزم `healthcheck` استفاده شود.

۱۰.۳ – جمع‌بندی hostname ها (راهنمای جامع آدرس‌دهی)

از مبدأ	به مقصد	/ hostname آدرس	مثال کانکشن استرینگ
برنامه روی سیستم میزبان	کانتینر	localhost	@localhost:5432 (نیازمند port (binding)
کانتینر	کانتینر دیگر در همان شبکه Compose	نام سرویس	@db:5432
کانتینر	اینترنت خارجی	مستقیم	curl https://api.github.com
سرور لینوکس	کانتینر	localhost	مانند سیستم میزبان (با پورت مپ‌شده)

۱۰.۴ – پرچم‌های پورت: تفاوت‌های ظریف

ports:

- "5432:5432" # برای همه اینترفیس‌ها – معمولی
- "127.0.0.1:5433:5432" # فقط از خود سیستم (امن‌تر برای دیتابیس!)

و یک نکته امنیتی خوب: در production، دیتابیس اصلاً -p نمی‌خواد! چون فقط api بهش نیاز داره و در شبکه‌ی داخلی با اسم سرویس وصل می‌شه. پورت نکشیده = از بیرون غیرقابل دسترس = امن. (خیلی از سرورهای هک‌شده، دیتابیس‌ی بودن که پورت ۵۴۳۲ شون به اینترنت باز بود!)

۱۰.۵ – بدون compose هم می‌شه (شبکه‌ی دستی)

برای فهم عمیق‌تر – بدون compose، دو کانتینر روی یک شبکه:

```
$ docker network create mynet
$ docker run -d --name db --network mynet -e POSTGRES_PASSWORD=1 postgres:18
$ docker run -d --name api --network mynet -p 3000:3000 \
-e DATABASE_URL="postgres://postgres:1@db:5432/postgres" myapi:1.0
```

--network مشترک + اسم کانتینر به عنوان compose – hostname دقیقاً همین رو خودکار می‌کنه.

✓ جمع‌بندی فصل

- از میزبان → کانتینر: (-p) پورت-مپ‌شده: localhost
- کانتینر → کانتینر: localhost غلطه! اسم سرویس/کانتینر = hostname (DNS داخلی داکر)
- depends_on فقط ترتیب است، نه صبر برای آماده‌شدن – برنامه‌ها retry می‌زنند

- دیتابیس در production پورت نکش – فقط شبکه داخلی

تمرین فصل ۱۰

1. استک api+db فصل ۸ را بالا بیاور؛ از داخل `getent hosts db` api: (یا `ping db` در ایمیج‌هایی که دارند) – آی‌پی کانتینر db را ببین.
2. `docker network ls` و `network inspect` روی شبکه پروژه – اعضا را ببین.
3. از سیستم خودت به db وصل شو با `localhost:5432`؛ بعد داخل کانتینر api امتحان کن – `localhost` ECONNREFUSED را ببین! (دیگه می‌دونی چرا!!)
4. در `compose`، پورت db را به `127.0.0.1:5433:5432` تغییر بده و از سیستم وصل شو (`localhost:5433`).
5. سوال مفهومی: در production سرور، چرا پورت db را کلاً نمی‌کشیم؟

▼ جواب تمرین ۵

چون تنها مصرف‌کننده‌ی دیتابیس، api است که در همان شبکه‌ی `compose` با `db` `hostname` به آن می‌رسد. پورت نگشتن به میزبان یعنی دسترسی از اینترنت ممکن نیست → سطح حمله صفر. اگر روزی از بیرون نیاز شد، فقط به `127.0.0.1` مپ می‌کنیم و با SSH tunnel می‌رسیم بهش.

→ **فصل بعد:** همه را جمع می‌کنیم – استک کامل توسعه‌ی شرکتی: Next.js + Postgres + Prisma در داکر!

فصل ۱۱ – داکر در کار روزمره: استک کامل + Next.js

Postgres + Prisma

🎯 هدف: سناریوی واقعی تیم‌های مهندسی نرم‌افزار: اجرای دیتابیس در کانتینر ایزوله، اجرای فریم‌ورک وب (مانند Next.js) روی سیستم میزبان، و ارتباط ORM (مانند Prisma) بین آن‌ها.

۱۱.۱ – الگوی استاندارد: دیتابیس داکری، اپ لوکال

در اغلب پروژه‌های مدرن تحت وب: برنامه اصلی روی سیستم توسعه‌دهنده اجرا می‌شود (`npm run dev`) – برای استفاده از hot-reload سریع) اما دیتابیس‌ها و سرویس‌های جانبی در کانتینر داکر نگهداری می‌شوند (برای پرهیز از نصب سنگین سرویس‌ها روی سیستم‌عامل و تضمین یکسانی نسخه در تمام تیم). این یک الگوی استاندارد در صنعت نرم‌افزار است:



۱۱.۲ – راه‌اندازی در ۵ دقیقه

در ریشه پروژه، فایل `compose.yaml` را بسازید:

```
services:
  db:
    image: postgres:18
    environment:
      POSTGRES_PASSWORD: devpass
      POSTGRES_DB: myshop
    ports:
      - "127.0.0.1:5432:5432" # فقط دسترسی امن از سیستم میزبان
    volumes:
      - pgdata:/var/lib/postgresql/data

volumes:
  pgdata:
```

و فایل `.env` پروژه:

```
DATABASE_URL="postgresql://postgres:devpass@localhost:5432/myshop"
```

بالا بیاورید و ابزار مهاجرت پایگاه داده (Prisma) را متصل کنید:

```
$ docker compose up -d
$ sleep 3 && docker compose logs --tail 5 db # صبر برای آماده شدن دیتابیس

$ npx prisma migrate dev # ها روی دیتابیس کانتینری migration اجرای
$ npm run dev # اجرای برنامه - متصل به دیتابیس داکر
```

به این ترتیب یک محیط توسعه کامل، نسخه دار، قابل حذف و بازسازی سریع فراهم می شود؛ هر برنامه نویسی دیگری در تیم با دریافت پروژه تنها با دستور `docker compose up -d` تمام سرویس های لازم را آماده خواهد داشت.

🔑 همیشه این فایل `compose.yaml` و یک نمونه `.env.example` را در مخزن گیت پروژه کامیت کنید تا فرآیند onboarding اعضای جدید تیم فوری و بی دردسر باشد.

۱۱.۳ – گردش های روزمره (چیت شیت زنده)

کار	دستور
شروع روز کاری	<code>docker compose up -d</code>
ریست کامل دیتابیس توسعه	<code>docker compose down -v && docker compose up -d</code> <code>prisma migrate dev</code>
دیدن لاگ دیتابیس	<code>docker compose logs -f db</code>
ورود به کنسول psql بدون نصب ابزار در سیستم	<code>docker compose exec db psql -U postgres -d myshop</code>
چک وضعیت سرویس ها	<code>docker compose ps</code>
پایان روز کاری	<code>docker compose stop</code> (سریع تر از down – کانتینرها متوقف می ماند)

به دستور `exec db psql` دقت کنید: حتی اگر کلاینت PostgreSQL روی سیستم عامل میزبان نصب نباشد، کلاینت داخلی کانتینر مستقیماً اجرا می شود (مطابق آموزش فصل ۵).

۱۱.۴ – نکات کلیدی ارتباط با دیتابیس و داکر

۱) **خطای عدم اتصال در ابتدای کار (P1001 در Prisma):** همان طور که در فصل ۱۰ دیدیم، بالا آمدن کانتینر دیتابیس چند ثانیه زمان می برد تا سوکت های شبکه آماده پذیرش ارتباط شوند. همیشه ابتدا با `docker compose logs db` آماده بودن سرور را بررسی کنید یا از مکانیزم `healthcheck` استفاده کنید.

۲) **خطای اشغال بودن پورت («Port already allocated»):** اگر قبلاً دیتابیس روی سیستم میزبان پورت ۵۴۳۲ را اشغال کرده باشد، دو راه حل وجود دارد: متوقف کردن آن سرویس در سیستم، یا تغییر پورت بیرونی کانتینر مثلاً به 5433:5432 و

تنظیم آدرس کانکشن استرینگ به `localhost:5433` . برای پیدا کردن پروسه اشغال کننده پورت می توان از `netstat` - استفاده کرد. `ano`

۳) تنظیم صحیح متغیرهای آدرس دهی: به تفاوت آدرس اتصال دقت کنید: زمانی که برنامه روی هاست اجرا می شود از `localhost` استفاده می کند، اما اگر برنامه داخل شبکه کانتینرها باشد از نام سرویس (مانند `db`) استفاده می کند.

۱۱.۵ – پروژه ی نهایی فصل: خودت اینو بساز ⭐

تمرین جامع – یک `docker-compose.yml` با سه سرویس برای یک اپ جدی:

```
services:
  db:
    image: postgres:18
    environment:
      POSTGRES_PASSWORD: ${DB_PASSWORD}
      POSTGRES_DB: ${DB_NAME}
    volumes:
      - pgdata:/var/lib/postgresql/data
    ports:
      - "127.0.0.1:5432:5432"
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 5s
      timeout: 3s
      retries: 5

  redis:
    image: redis:7-alpine

  adminer:
    image: adminer
    ports:
      - "8081:8080"

volumes:
  pgdata:
```

چیزهای جدید این فایل:

مفهوم	کار
<code>healthcheck</code>	داکر خودش چک می کنه db «واقعاً» آماده است (<code>pg_isready</code>) - و <code>depends_on: condition:</code> <code>service_healthy</code> می تونه بهش تکیه کنه!
<code>adminer</code>	یک GUI وب برای دیتابیس - باز کن <code>localhost:8081</code>
<code>\${DB_PASSWORD}</code>	از فایل <code>.env</code> کنار <code>compose</code> می آید (فصل ۸)

بزن `docker compose up -d` - سه سرویس: دیتابیس نسخه دار + ماندگار، ردیس، و یک داشبورد گرافیکی. به این می گن

محیط توسعه ی خودتمون شده 🤖

✓ جمع‌بندی فصل

- الگوی رایج: دیتابیس/سرویس‌ها داکری، اپ لوکال با hot-reload
- استارت یک پروژه‌ی تیمی: dev → migrate → compose up → clone
- exec db psql = ابزارهای داخل کانتینر؛ down -v = ریست کامل توسعه
- healthcheck = «واقعاً آماده»؛ پورت‌ها فقط برای میزبان (127.0.0.1)

📝 تمرین فصل ۱۱

1. استک دوسرویس (دیتابیس در داکر + برنامه لوکال) را کامل راه بینداز – دستورات migration را اجرا کن و صفحه اول برنامه را تست کن.
2. ریست کامل توسعه: prisma migrate dev → up → down -v → داده‌ی اولیه (seed) را تزریق کن (prisma db seed).
3. Adminer را بالا بیاور و با آن به دیتابیس وصل شو (host: db یا از سیستم: localhost:5432 ؟ هر دو را تست کن!).
4. healthcheck را اضافه کن و docker compose ps بزن – ستون health را ببین (starting → healthy).
5. فایل‌های compose.yaml و .env.example را در پروژه کامیت کن (پیام: chore: add docker compose dev stack) و به مخزن push کن.

▼ نکته تمرین ۳

Adminer داخل کانتینر است؛ host اش باید db باشد (همان شبکه compose) – از سیستم هم localhost:5432 کار می‌کند چون پورت مپ شده. دو مسیر، دو زاویه – همان فصل ۱۰!

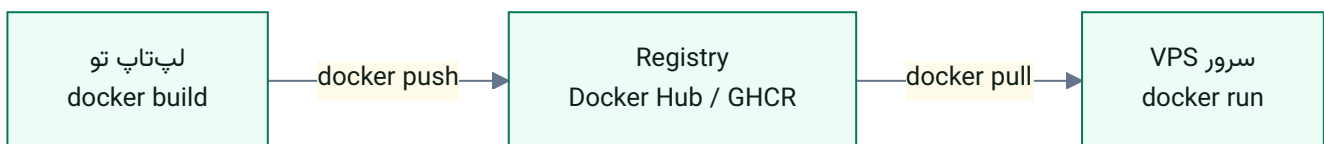
➔ فصل بعد: از محیط لوکال تا پروداکشن – push ایمیج و دپلوی روی سرور.

فصل ۱۲ – push و دیپلوی: از لپ‌تاپ به سرور

🎯 هدف: حلقه رو ببندیم: ایمج ساخته‌شده رو به رجیستری push کن، روی یک VPS واقعی دیپلوی کن، و نگهداری/امنیت پایه رو یاد بگیر. بعد از این فصل، «فول‌استک دیپلوی‌شده» واقعاً معنا داره.

۱۲.۱ – چرخه توزیع: build → push → pull → run

همون مدل npm: پکیج رو publish می‌کنی، هر کسی (حتی سرور) install می‌کنه:



۱۲.۲ – push به Docker Hub

```
$ docker login # با حساب کاربری رجیستری
$ docker build -t USERNAME/myapi:1.0 . # !باشد اسم owner/name باید
$ docker tag myapi:1.0 USERNAME/myapi:1.0 # یا تگ مجدد ایمج موجود
$ docker push USERNAME/myapi:1.0
```

(مشابه انتشار پکیج‌ها – ساختار `owner/name` الزامی است.) و روی هر سرور یا سیستمی در دنیا:

```
$ docker run -d -p 3000:3000 USERNAME/myapi:1.0
```

💡 تگ‌گذاری معنادار: `USERNAME/myapi:1.0` و `USERNAME/myapi:latest` – ارسال هر دو تگ توصیه می‌شود (latest برای دسترسی سریع، نسخه برای دقت و ثبات). دستور `docker tag` یک پرچسب جدید روی همان ایمج می‌زند و نیازی به بیلد مجدد نیست.

۱۲.۳ – گزینه‌ی مدرن: GHCR (رجیستری گیت‌هاب)

استفاده از GHCR / GitHub Packages به ویژه برای مخازن گیت‌هاب بسیار یکپارچه است:

```
$ echo $GITHUB_TOKEN | docker login ghcr.io -u USERNAME --password-stdin
$ docker tag myapi:1.0 ghcr.io/USERNAME/myapi:1.0
$ docker push ghcr.io/USERNAME/myapi:1.0
```

و خودکارسازی: در GitHub Actions، می‌توان یک پایپ‌لاین CI/CD نوشت که با هر Git Tag یا انتشار نسخه جدید، ایمج را بیلد کرده و به GHCR ارسال کند.

۱۲.۴ – دپلوی روی VPS (سرور لینوکسی)

اتصال به سرور لینوکسی از طریق SSH:

```
# روی سرور - یک بار:
$ ssh root@203.0.113.10
# نصب داکر (اسکریپت رسمی):
curl -fsSL https://get.docker.com | sh
docker --version

# دپلوی:
$ docker pull ghcr.io/USERNAME/myapi:1.0
$ docker run -d --name api \
  -p 80:3000 \
  --restart unless-stopped \
  -e DATABASE_URL="..." \
  ghcr.io/USERNAME/myapi:1.0

$ curl http://203.0.113.10 # اپلیکیشن روی سرور آماده پاسخگویی است 🚀
```

- `--restart unless-stopped` = بعد از کرش یا ری‌استارت سرور، کانٹینر خودبه‌خود روشن می‌شود (فصل ۳)
- `-p 80:3000` = پورت استاندارد ۸۰ وب به پورت کانٹینر متصل می‌شود (یا در پشت Nginx قرار می‌گیرد)

به‌روزرسانی نسخه (چرخه‌ی همیشگی):

```
# بعد از انتشار نسخه جدید:
$ docker pull ghcr.io/USERNAME/myapi:1.1
$ docker stop api && docker rm api
$ docker run -d --name api -p 80:3000 --restart unless-stopped \
  -e DATABASE_URL="..." ghcr.io/USERNAME/myapi:1.1
```

(این چرخه دستی برای شروع ساده‌ترین حالت است؛ در مراحل پیشرفته‌تر با ابزارهایی مثل Watchtower یا وب‌هوک‌ها خودکارسازی می‌شود.)

استک با compose روی سرور:

اگر فایل compose آماده کرده‌اید، روی سرور به سادگی قابل اجراست:

```
$ git clone https://github.com/USERNAME/myproj.git && cd myproj
$ nano .env # تنظیم متغیرهای محیطی محیط پروداکشن
$ docker compose up -d --build
```

داده‌های دیتابیس در production هم با Volume داکر ذخیره می‌شوند – و فرآیند **بک‌آپ‌گیری** منظم (آموزش داده‌شده در فصل ۹) الزامی است.

۱۲.۵ – بهداشت سرور: پاکسازی

داکر روی سرور با نسخه‌های قدیمی ایمیج‌ها و کش بیلد ممکن است فضای دیسک را پر کند:

```
$ docker system df          # گزارش فضای مصرفی داکر
$ docker system prune       # حذف کانتینرهای متوقف + شبکه‌های بلااستفاده + کش بیلد
$ docker system prune -a    # همراه با ایمیج‌های استفاده‌نشده (با احتیاط) ⚠
```

یک کار دوره‌ای یا cron job برای پاکسازی ایمیج‌های بی‌استفاده به سلامت سرور کمک شایانی می‌کند.

۱۲.۶ – امنیت پایه (چک‌لیست):

- [] **کلیدها و اطلاعات حساس در env، نه داخل ایمیج:** هرگز `ENV SECRET=...` در Dockerfile قرار نگیرد؛ همیشه از فایل‌های `.env`، ابزارهای مدیریت secret یا فلگ `-e` استفاده شود
- [] فایل‌های حساس `.env` هرگز به ایمیج یا مخزن گیت اضافه نشوند (تنظیم دقیق `.dockerignore` و `.gitignore`)
- [] استفاده از تگ‌های نسخه‌دار دقیق به جای `latest` در سرور پروداکشن
- [] دیتابیس بدون پورت پابلیک باز اجرا شود (فقط در شبکه داخلی داکر، مطابق فصل ۱۰)
- [] کانتینرها در صورت امکان با کاربر غیر روت اجرا شوند (`USER node` در Dockerfile برای پروژه‌های نود)
- [] ایمیج پایه را مرتب pull/بازسازی کن (آپدیت‌های امنیتی پایه)
- [] پورت‌های management (مثل ۲۳۷۵ داکر) بسته باشند

✓ جمع‌بندی فصل

- چرخه: `pull` → `build -t owner/name:tag` → سرور `run -d -p --restart`
- Docker Hub یا GHCR؛ تگ‌گذاری معنادار (نسخه + latest)
- دپلوی ساده VPS: داکر نصب → `run` با `restart`؛ استک‌ها با `compose up`
- `system prune` برای بهداشت فضا؛ امنیت: رازها با `env`، تگ دقیق، دیتابیس بی‌پورت

📄 تمرین فصل ۱۲

- ایمیج myapi را با تگ‌های `1.0` و `latest` بساز و به Hub (یا GHCR) push کن.
- ایمیج pushed را حذف از لوکال (`docker rmi`) و دوباره `docker run` – از رجیستری pull شد!

3. (اگر VPS داری 🌟) کل چرخه: pull و run و curl به IP سرور. (بدون VPS: شبیه‌سازی – pull روی سیستم خودت).

4. `docker system df` و بعد `prune` – قبل و بعد را مقایسه کن.

5. چک‌لیست امنیتی را روی یک Dockerfile نمونه یا پروژه واقعی بررسی کن – چند مورد از موارد چک‌لیست رعایت شده است؟

▼ نکته تمرین ۱

```
docker build -t USERNAME/myapi:1.0 -t USERNAME/myapi:latest .  
docker push USERNAME/myapi:1.0  
docker push USERNAME/myapi:latest
```

دو تگ = دو push (هر کدام یک اشاره‌گر به همان لایه‌ها – دومی تقریباً فوری است چون لایه‌ها آپلود شده‌اند).

➡ **فصل آخر:** چیت‌شیت کامل و گلاساری – مرجع سریع دستورات داکر.

فصل ۱۳ – چیت‌شیت، گلاساری و ارورهای کلاسیک

🎯 هدف: مرجع مرور سریع – چاپش کن کنار مانیتورت.

۱۳.۱ – چیت‌شیت دستورات

```
# == ایمیج (فصل ۴) ==
docker pull postgres:18
docker images / docker rmi <img>
docker build -t name:1.0 .
docker history <img>
docker system df / prune

# == کانتنیتر (فصل ۳) ==
docker run -d -p 8080:80 --name web nginx
# متغیر VAR=val ↑ -e میزبان:کانتنیتر ↑
docker ps / docker ps -a
docker logs -f --tail 50 web
docker stop/start/restart/rm web
docker exec -it web bash
docker cp web:/path ./
docker container prune

# == Compose (فصل ۸) ==
docker compose up -d [--build]
docker compose ps / logs -f api
docker compose exec api sh
docker compose down / down -v
docker compose config

# == داده (فصل ۹) ==
docker volume ls / rm / inspect
-v pgdata:/var/lib/postgresql/data
-v ./src:/app/src

# == شبکه (فصل ۱۰) ==
docker network ls / inspect <net>
اسم سرویس = hostname
localhost = کانتنیتر #

# == توزیع (فصل ۱۲) ==
docker login
docker tag myapi:1.0 user/myapi:1.0
docker push user/myapi:1.0
docker run -d --restart unless-stopped -p 80:3000 user/myapi:1.0
```

دانلود (تگ دقیق!)
لیست / حذف
Dockerfile ساخت از
لایه‌ها
فضا / پاکسازی

در حال اجرا / همه
لاگ زنده
چرخه عمر
(alpine: sh) ورود
کپی دوطرفه
جاروی متوقف‌ها

بالا بیاور
وضعیت / لاگ
ورود به سرویس
(⚠) توقف / + حذف داده
env نهایی با YAML

named (production)
bind (توسعه)

کانتنیتر→کانتنیتر
! غلط localhost فقط برای میزبان

کار	دستور
ایمیج پایه (همیشه اول؛ alpine برای production)	<code>FROM node:22-alpine</code>
پوشه کاری داخل کانتینر	<code>WORKDIR /app</code>
آوردن فایل‌ها / از مرحله دیگر	<code>COPY . . /</code> / <code>COPY --from=x</code>
اجرا حین build (در ایمیج ثبت می‌شود)	<code>RUN npm ci</code>
متغیر محیطی	<code>ENV NODE_ENV=production</code>
مستندسازی (باز کردن واقعی با -p)	<code>EXPOSE 3000</code>
پروژه اصلی حین run	<code>CMD ["node", "server.js"]</code>

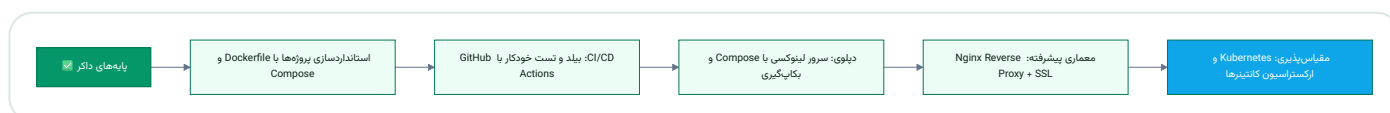
قانون‌های Dockerfile: dependency ها بالای سورس (کش!)؛ multi-stage برای production. dockerignore همیشه؛ پاک‌سازی در همان `&&` (RUN).

انگلیسی	فارسی	یک جمله
Container	کانتینر	نمونه‌ی در حال اجرای ایمیج – ایزوله و سبک
Image	ایمیج	قالب فقطخواندنی حاوی اپ و محیطش
Registry	رجیستری	انبار ایمیج‌ها (Docker Hub، GHCR)
Layer	لایه	پله‌ی فقطخواندنی ایمیج؛ پایه‌ی کش
Tag	تگ	نسخه‌ی ایمیج (postgres:18)
Dockerfile	–	دستور پخت ایمیج
Compose	–	تعریف چند سرویس در یک YAML
Volume	ولوم / حجم	فضای ماندگار مستقل از کانتینر
Bind Mount	اتصال پوشه	اتصال پوشه‌ای از سیستم میزبان به داخل کانتینر (برای توسعه)
Port Mapping	نگاشت پورت	میزبان: کانتینر -p
Build Context	زمینه build	پوشه‌ای که به build داده می‌شود (.)
Multi-stage	چندمرحله‌ای	build جدا از runtime برای ایمیج سبک
Daemon	دیمن	سرویس پس‌زمینه‌ی داکر (Engine)
Healthcheck	بررسی سلامت	«واقعاً» آماده است؟ (pg_isready)
WSL2	–	لینوکس زیر Docker Desktop در ویندوز

۱۳.۴ – دوازده ارور کلاسیک و ترجمه

ارور	یعنی	علاج
port is already allocated	پورت میزبان اشغال	استفاده از پورت دیگر یا بستن پروسه اشغال‌کننده با <code>netstat / taskkill</code>
Cannot connect to Docker daemon	Engine خاموش	Docker Desktop یا سرویس Docker را اجرا کنید
manifest unknown / pull access denied	ایمیج/تگ غلط	نام و تگ را بررسی کنید؛ در صورت خصوصی بودن لاگین کنید (<code>docker login</code>)
ECONNREFUSED به دیتابیس از اپ کانتینری	استفاده از localhost داخل کانتینر	نام سرویس به عنوان هاست قرار گیرد: <code>@db:5432</code> (فصل ۱۰)
P1001 در Prisma	db هنوز آماده نشده	<code>docker compose logs db</code> و صبر برای راه‌اندازی یا استفاده از <code>healthcheck</code>
exec: bash: not found	ایمیج مبتنی بر alpine است	استفاده از <code>sh</code> به جای <code>bash</code>
Exited کانتینر بلافاصله	پروسه اصلی متوقف شد	بررسی خروجی با <code>docker logs</code> (معمولاً ارور سینتکس یا متغیرهای محیطی)
داده‌ی دیتابیس پاک شد	volume تعریف نشده بود	تعریف و مانت Volume پایدار با <code>-v pgdata:...</code> (فصل ۹)
حجم ایمج خیلی بزرگ شد	ابزارهای بیلد و فایل‌های اضافه داخل ایمج مانده	Multi-stage build + حذف فایل‌های غیرضروری در <code>.dockerignore</code> (فصل ۷)
تغییر کد در پروژه اثر نکرد	ایمیج مجدداً بیلد نشده	اجرای <code>docker compose up -d --build</code>
no space left on device	دیسک پر شده	بررسی با <code>docker system df</code> و پاکسازی با <code>docker system prune</code>
خطای اجرای پکیج‌های نیتیو (node_modules)	کپی مستقیم پوشه لوکال هاست به لینوکس	نادیده گرفتن <code>node_modules</code> در <code>.dockerignore</code> و نصب داخل کانتینر (فصل ۶)

۱۳.۵ – نقشه راه پس از این دوره



سه توصیه‌ی پایانی برای تسلط کامل:

1. **تمرین روی پروژه‌های واقعی:** در اولین فرصت، استک توسعه (مانند فصل ۱۱) را به پروژه‌های فعلی خود اضافه کنید و با ایجاد `compose.yaml` و `.env.example`، مدیریت دیتابیس و محیط را کانتینری کنید.
 2. **استفاده مستمر از راهنمای خط فرمان:** تمام دستورات و سوییچ‌ها با دستورات `docker` و `docker --help` و `compose --help` همواره در دسترس هستند و درک ساختار دستورات مهم‌تر از حفظ کردن آن‌هاست.
 3. **مستندات رسمی داکر:** سایت docs.docker.com بهترین مرجع برای جزئیات فنی دقیق و قابلیت‌های پیشرفته‌تر است که با تسلط بر مفاهیم پایه‌ای این دوره، مطالعه آن بسیار آسان خواهد بود.
- با تسلط بر این مفاهیم، توانایی بسته‌بندی، ایزوله‌سازی، تست و دیپلوی پروژه‌های نرم‌افزاری به صورت استاندارد در داکر به دست آمده است. 🚀🐳